

Increasing Awareness in Distance-Based Learning

Alfonso Iodice D'Enza

2026-09-16

31st
Edition

2026 ICAMD
2nd Edition

INTERNATIONAL CONFERENCE ON STATISTICS AND DATA SCIENCE
14-18 September , Casablanca, MOROCCO

A collaborative project

Italy, the Netherlands, Greece, and the United States



Increasing awareness in distance construction

A line of work for distance-based learning with mixed data

Scale/type-aware

Additivity, commensurability, and bias in mixed data

Association-aware

Redundancy, correlations, and categorical associations

Interaction-aware

Continuous–categorical relationships and local neighbourhood structure

Response-aware

Supervised neighbourhoods and prediction-oriented distances



Building distance-based pipelines: **manydist**

Distance construction, diagnostics, and reproducible learning workflows

1. Learning from distances

Learning from mixed-type data

Many statistical learning methods rely, explicitly or implicitly, on **comparing observations**.

Terminology alert!

I will often use **distance** as shorthand for pairwise dissimilarity.

Some measures discussed here are not metrics in the strict mathematical sense, but they all quantify how different two observations are.

- If the data are mixed-type, then comparison is **not straightforward**:

Many learning methods can operate directly on a dissimilarity matrix.

Dimension reduction

Multidimensional scaling (MDS)¹

Clustering methods

Hierarchical clustering (HC), PAM², and spectral clustering³

Nearest-neighbour prediction

Nearest-neighbour classification and nearest-neighbour averaging for regression

building pairwise dissimilarities: intuition

2 continuous variables: add up by-variable (absolute value or squared) differences

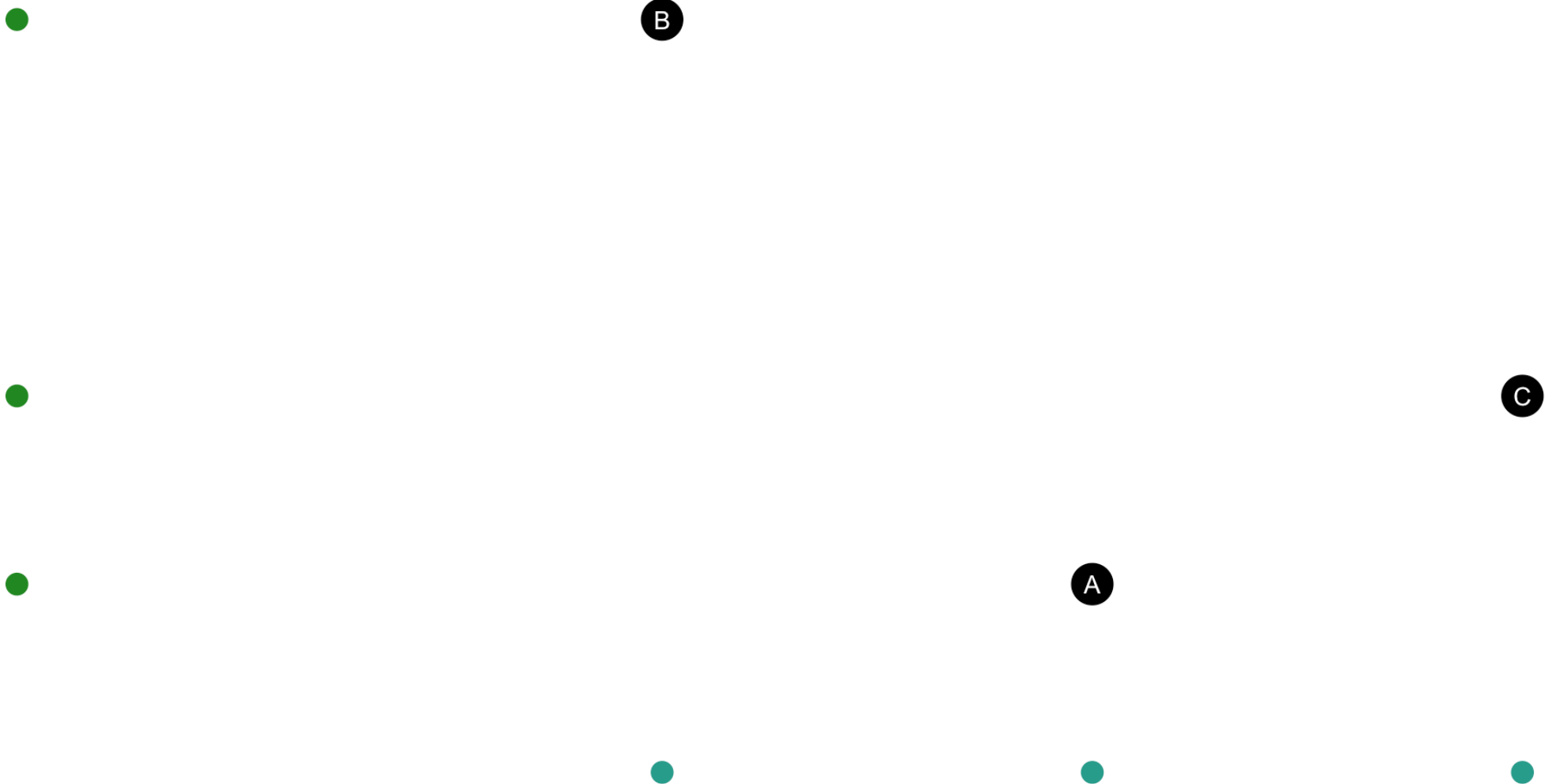
B

C

A

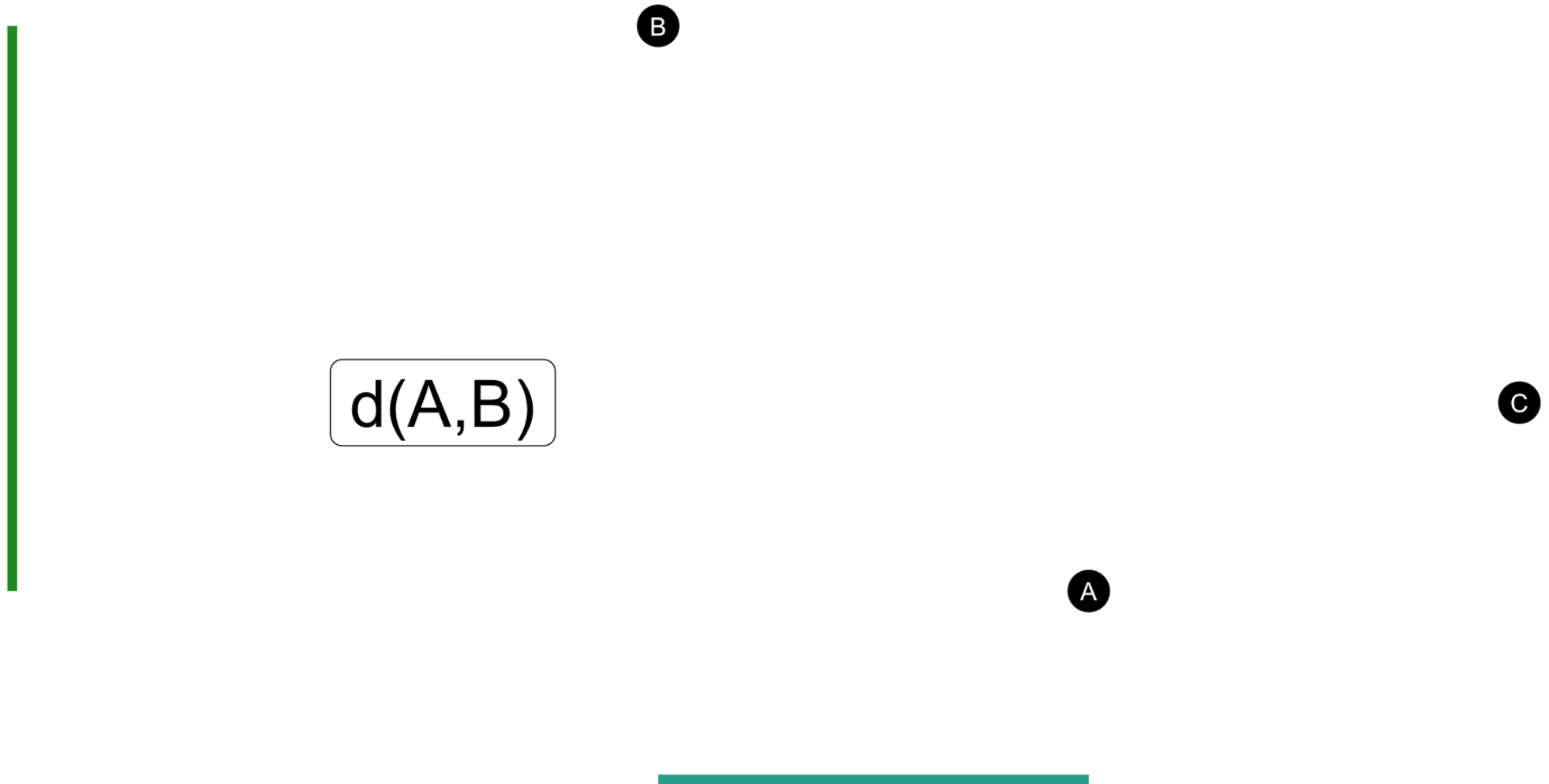
building pairwise dissimilarities: intuition

2 continuous variables: add up by-variable (absolute value or squared) differences



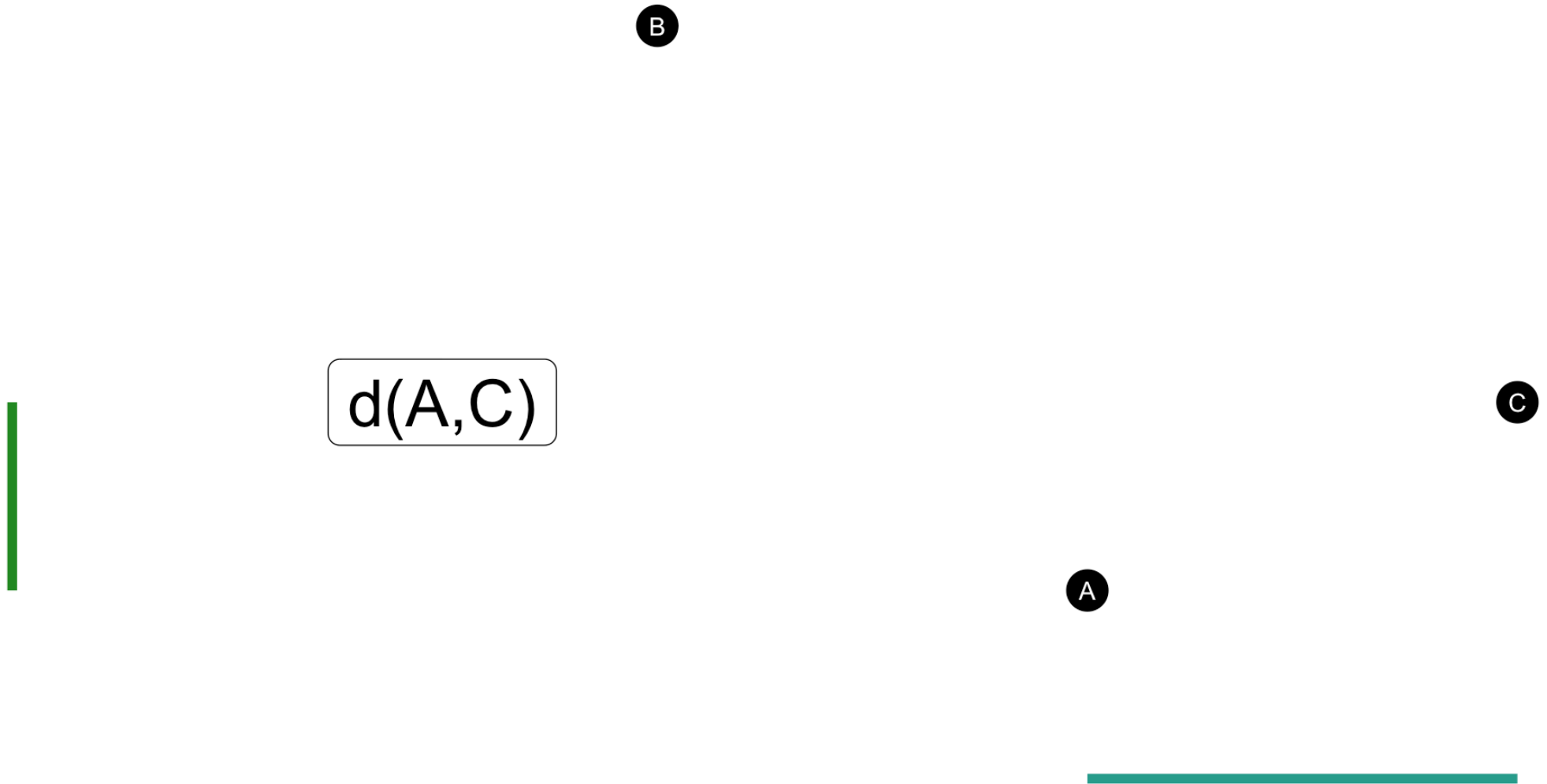
building pairwise dissimilarities: intuition

2 continuous variables: add up by-variable (absolute value or squared) differences



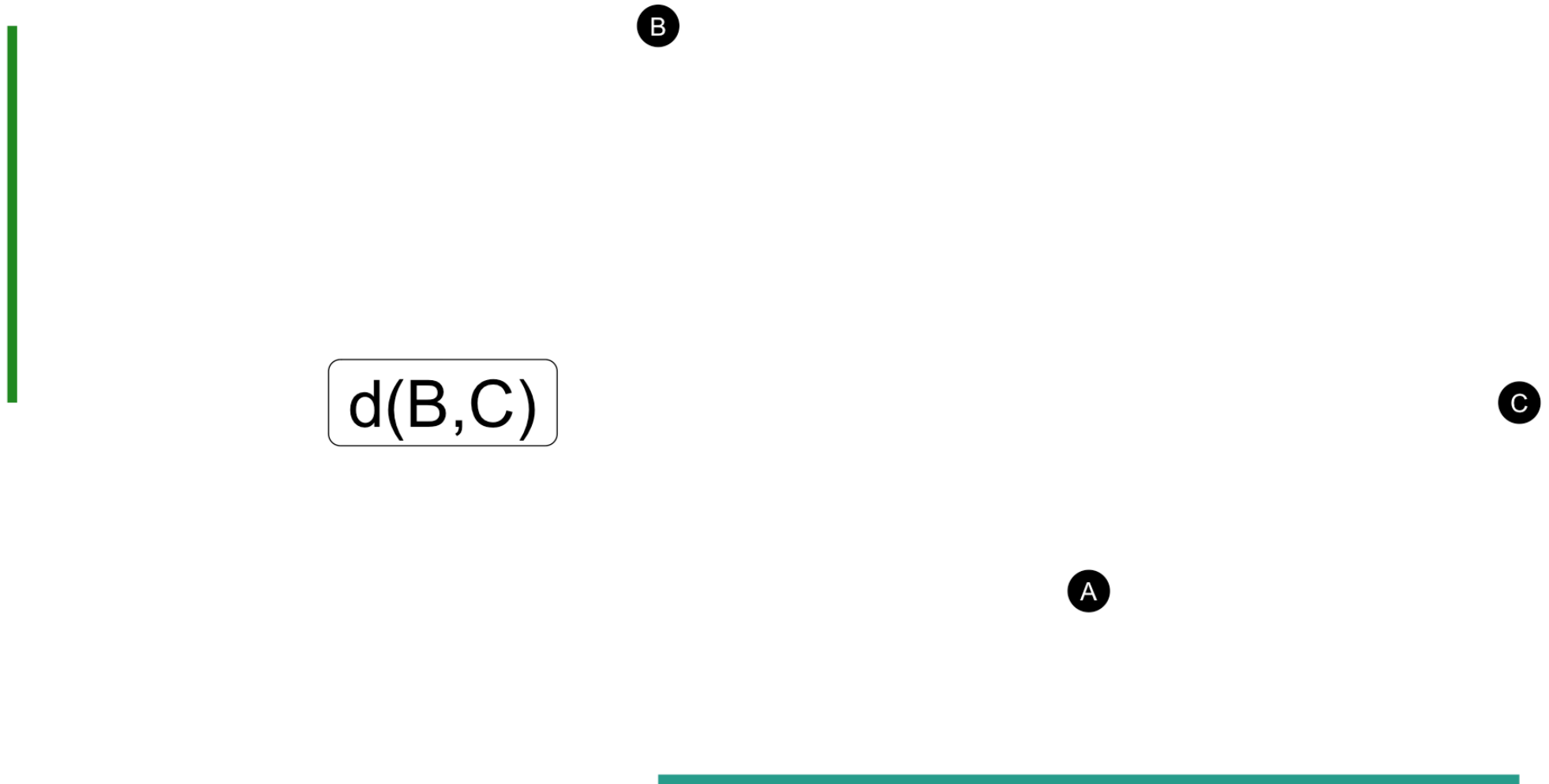
building pairwise dissimilarities: intuition

2 continuous variables: add up by-variable (absolute value or squared) differences



building pairwise dissimilarities: intuition

2 continuous variables: add up by-variable (absolute value or squared) differences



building pairwise dissimilarities: intuition

2 continuous and 1 categorical variables

B

C

A

building pairwise dissimilarities: intuition

one might consider purple and blue *closer* than e.g. purple and yellow

B

C

A

2. Desirable properties and general setup

Desirable properties¹

Multivariate Additivity

Let $\mathbf{x}_i = (x_{i1}, \dots, x_{iQ})$ denote a Q –dimensional vector. A distance function $d(\mathbf{x}_i, \mathbf{x}_\ell)$ between observations i and ℓ is multivariate **additive** if

$$d(\mathbf{x}_i, \mathbf{x}_\ell) = \sum_{j=1}^Q d_j(\mathbf{x}_i, \mathbf{x}_\ell),$$

where $d_j(\mathbf{x}_i, \mathbf{x}_\ell)$ denotes the j –th variable specific distance.

- **Manhattan** distance satisfies the additivity property; the **Euclidean** distance does not

Desirable properties¹

If additivity holds, by-variable distances are added together: they should be on equivalent scales

Commensurability

Let $\mathbf{X}_i = (X_{i1}, \dots, X_{iQ})$ denote a Q –dimensional random variable corresponding to an observation i . Furthermore, let d_j denote the distance function corresponding to the j –th variable.

We have **commensurability** if, for all j , and $i \neq \ell$,

$$E[d_j(X_{ij}, X_{\ell j})] = c,$$

where c is some constant.

Mixed-data setup

a mixed data set

- n observations described by Q variables, Q_n numerical and Q_c categorical
- the $n \times Q$ data matrix $\mathbf{X} = [\mathbf{X}_n, \mathbf{X}_c]$ is column-wise partitioned

A formulation for mixed distance between observations i and ℓ :

$$\begin{aligned} d(\mathbf{x}_i, \mathbf{x}_\ell) &= \sum_{j_n=1}^{Q_n} d_{j_n}(\mathbf{x}_i^{(n)}, \mathbf{x}_\ell^{(n)}) + \sum_{j_c=1}^{Q_c} d_{j_c}(\mathbf{x}_i^{(c)}, \mathbf{x}_\ell^{(c)}) = \\ &= \sum_{j_n=1}^{Q_n} w_{j_n} \delta_{j_n}^{(n)}(\mathbf{x}_i^{(n)}, \mathbf{x}_\ell^{(n)}) + \sum_{j_c=1}^{Q_c} w_{j_c} \delta_{j_c}^{(c)}(\mathbf{x}_i^{(c)}, \mathbf{x}_\ell^{(c)}) \end{aligned}$$

numeric case

- $\delta_{j_n}^{(n)}$ is a function quantifying the dissimilarity between observations on the j_n -th numerical variable
- w_{j_n} is a weight for the j_n -th variable.

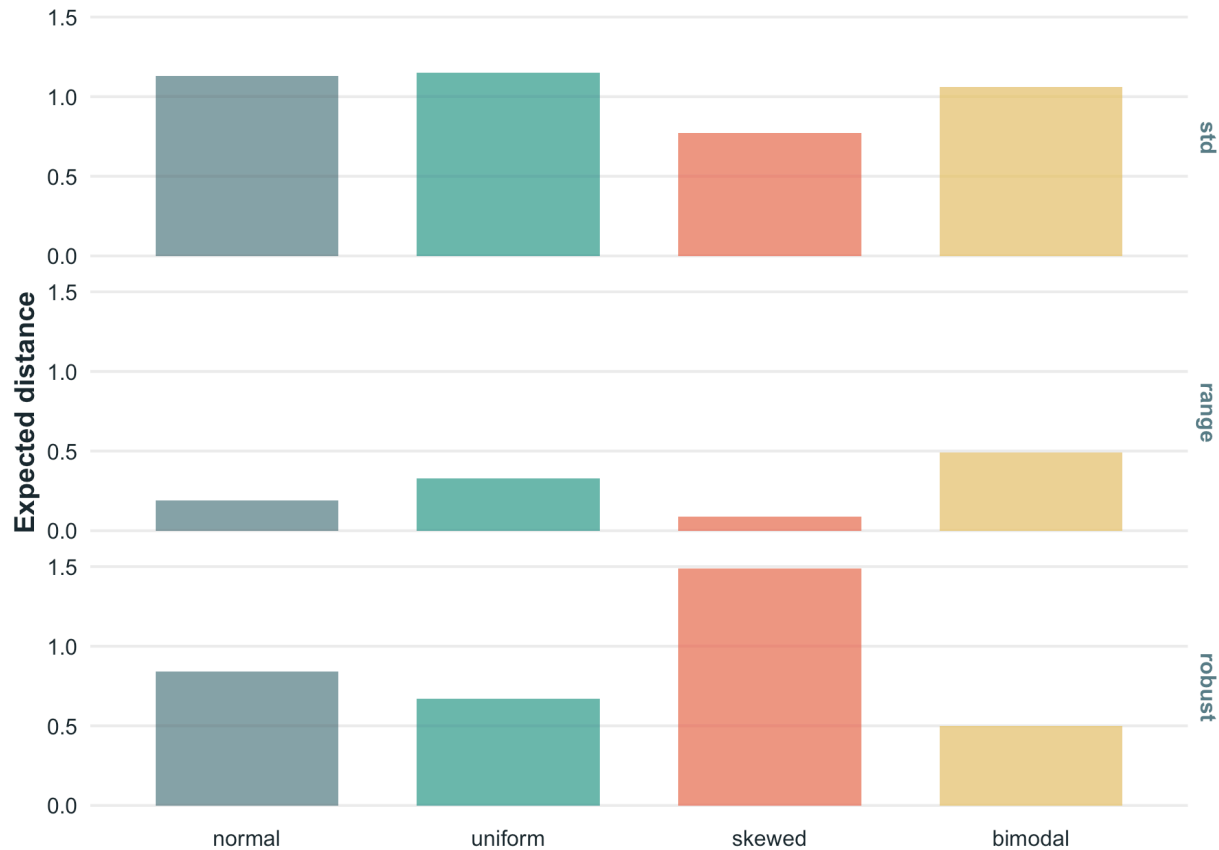
categorical case

dissimilarity between the categories chosen by subjects i and ℓ for categorical variable j_c

- w_{j_c} is a weight for the j_c -th variable

Scale/type-induced biases

Distributions, scaling and bias: the numeric case

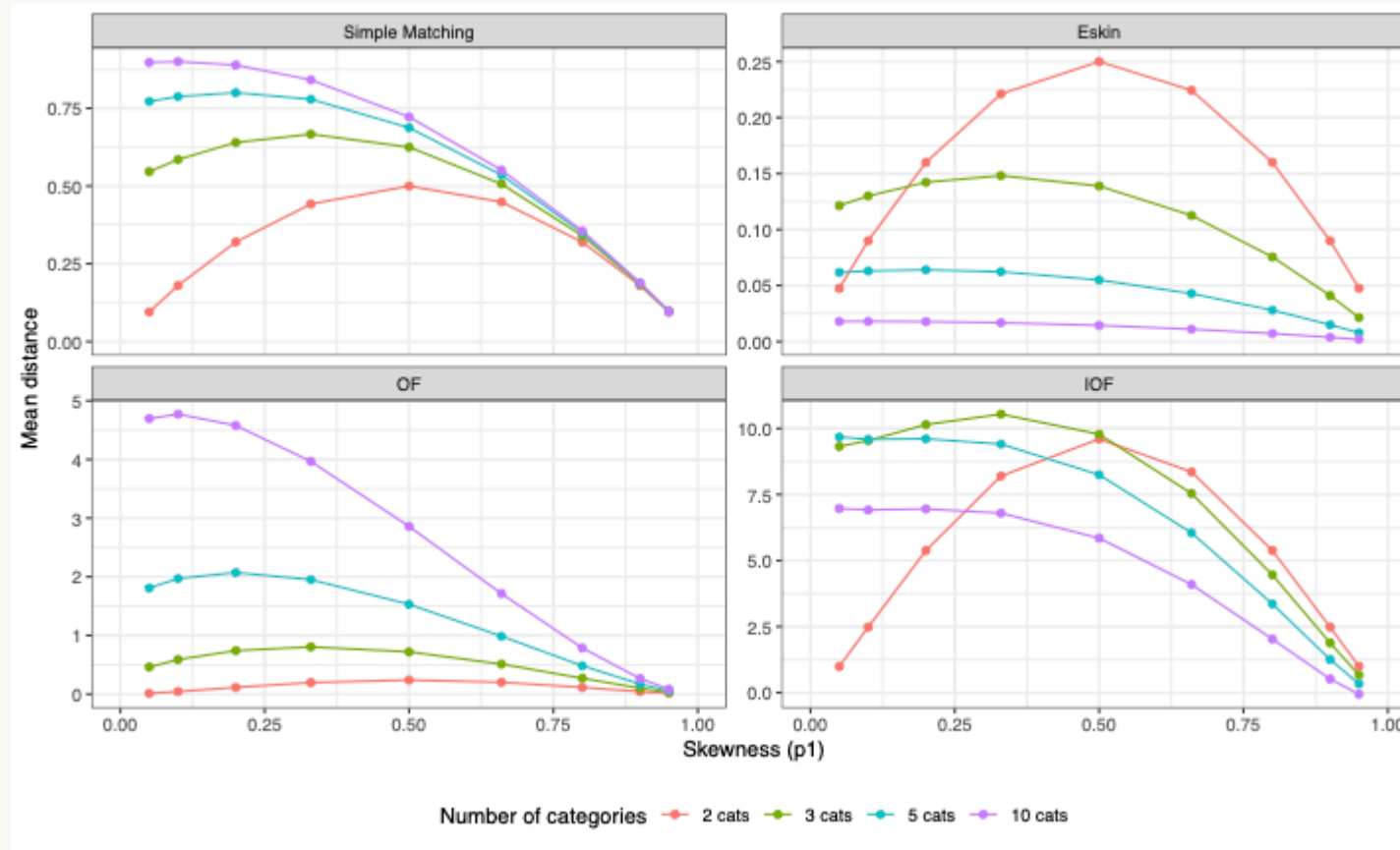


as long as variables have the same underlying distribution and scaling, commensurability holds

- skewed variables may be under- or over-contributing to the distance, depending on the scaling (range and robust, respectively)
 - the contribution of a variable to the overall distance may be **biased**

distributions, scaling and bias: the categorical case

skewed frequency distribution



- $q \in \{2, 3, 5, 10\}$
- $p_1 \in \{0.05, 0.1, 0.2, 0.33, 0.5, 0.66, 0.8, 0.9, 0.95\}$
- $p_j = (1 - p_1)/(q - 1)$, with $j = 2, \dots, q$,

The expected distance increases with the heterogeneity of the distribution and with the number of categories

Independence-based distances

Independence-based pairwise distance

No inter-variable relations are considered.

- in the continuous case: **Euclidean** or **Manhattan** distances
- in the categorical case: **Hamming** / matching distance, among many others
- in the mixed-data case: **Gower** dissimilarity index

variable contributions may be balanced, but still treated as separate sources of information

Beyond commensurability

commensurability makes variable contributions comparable across **scales** and data **types**.

- If variables are correlated or associated, the same information may contribute repeatedly to the distance: **redundancy**

the next step is to account for the **structure** among variables

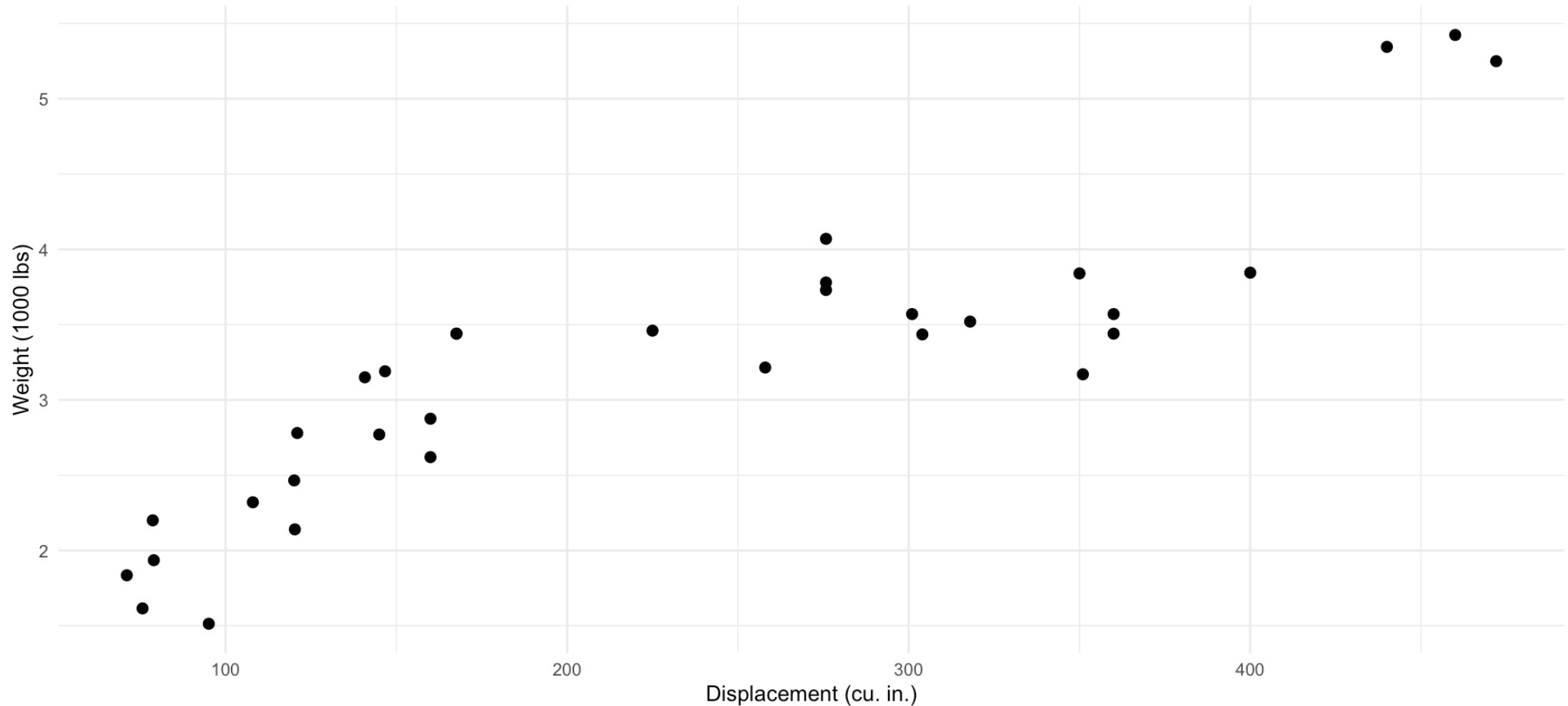
3. Association-aware distances

by variable differences: independence-based

- When variables are correlated or associated, shared information is effectively counted multiple times
- inflated dissimilarities may cause potential distortions in downstream unsupervised learning tasks.

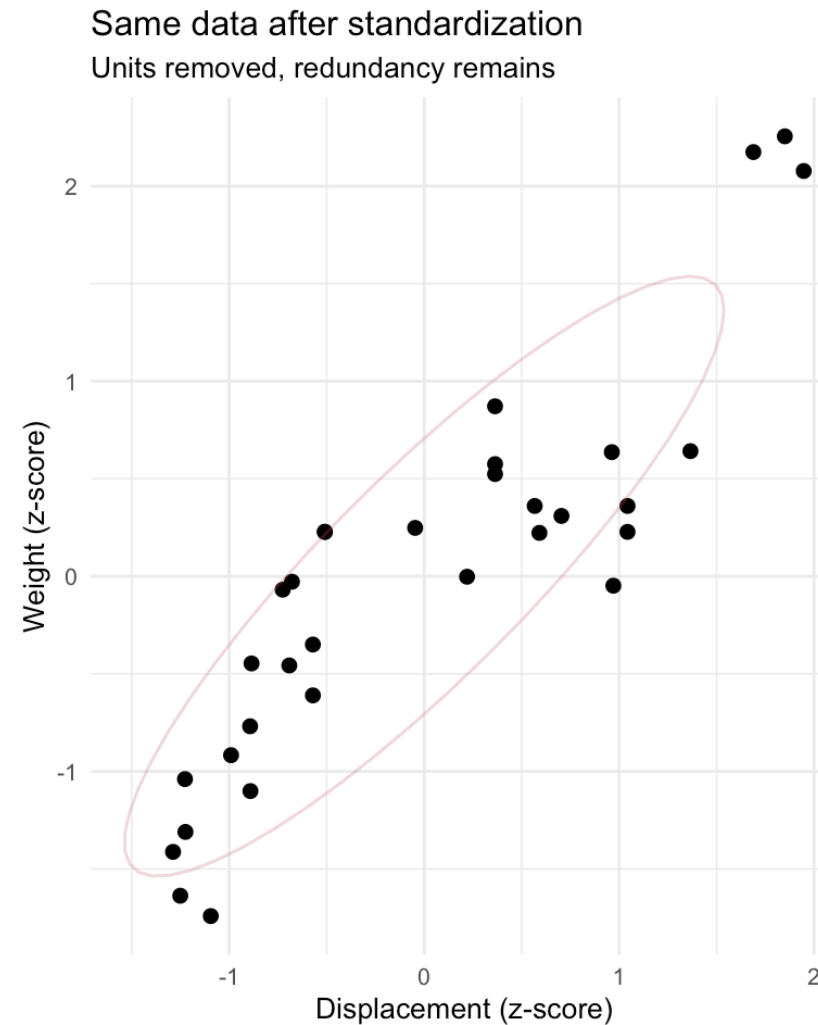
Redundant continuous variables: displacement and weight

Larger cars tend to have both higher displacement and weight



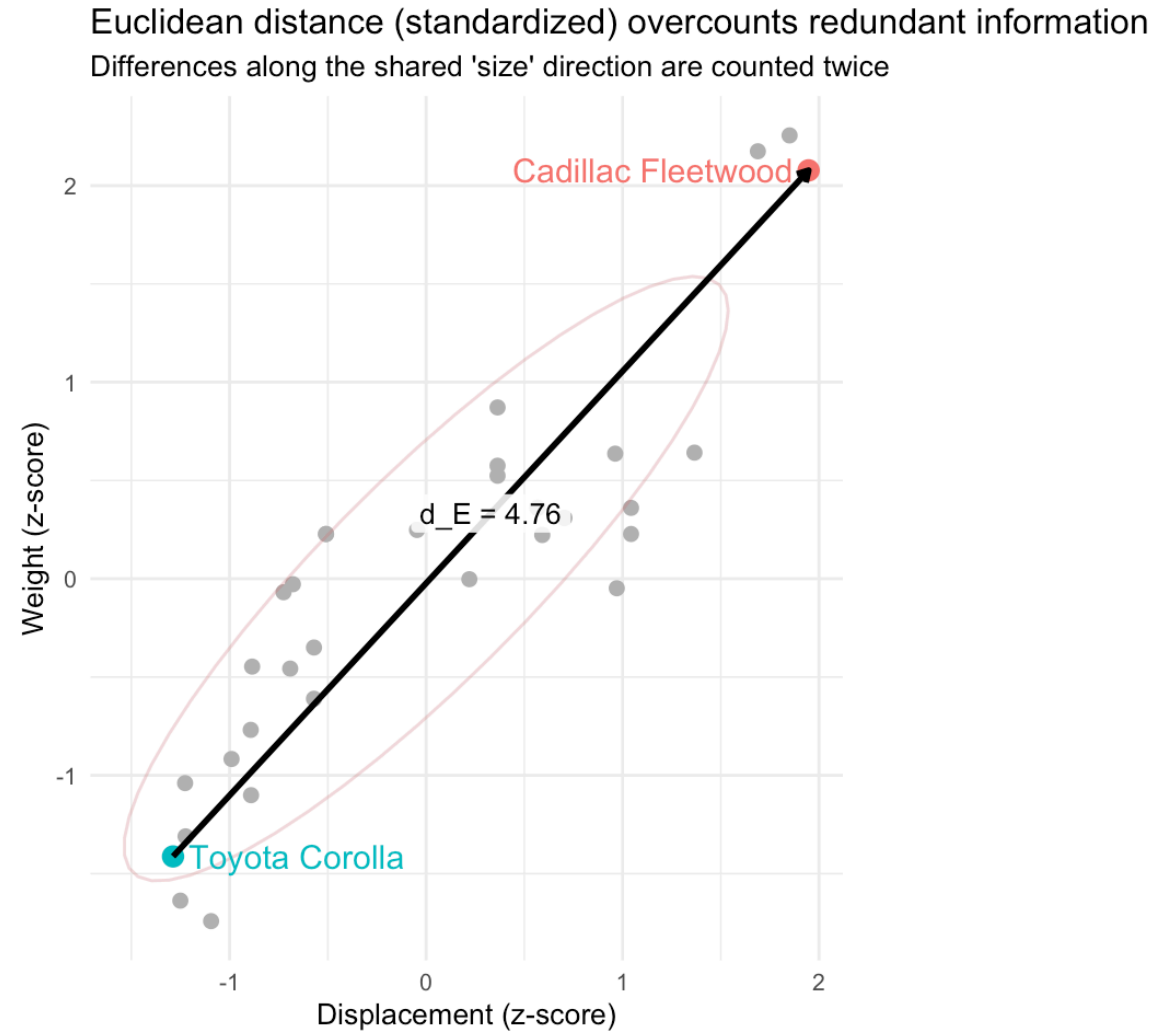
by variable differences: **independence-based**

- When variables are correlated or associated, shared information is effectively counted multiple times
- inflated dissimilarities may cause potential distortions in downstream unsupervised learning tasks.



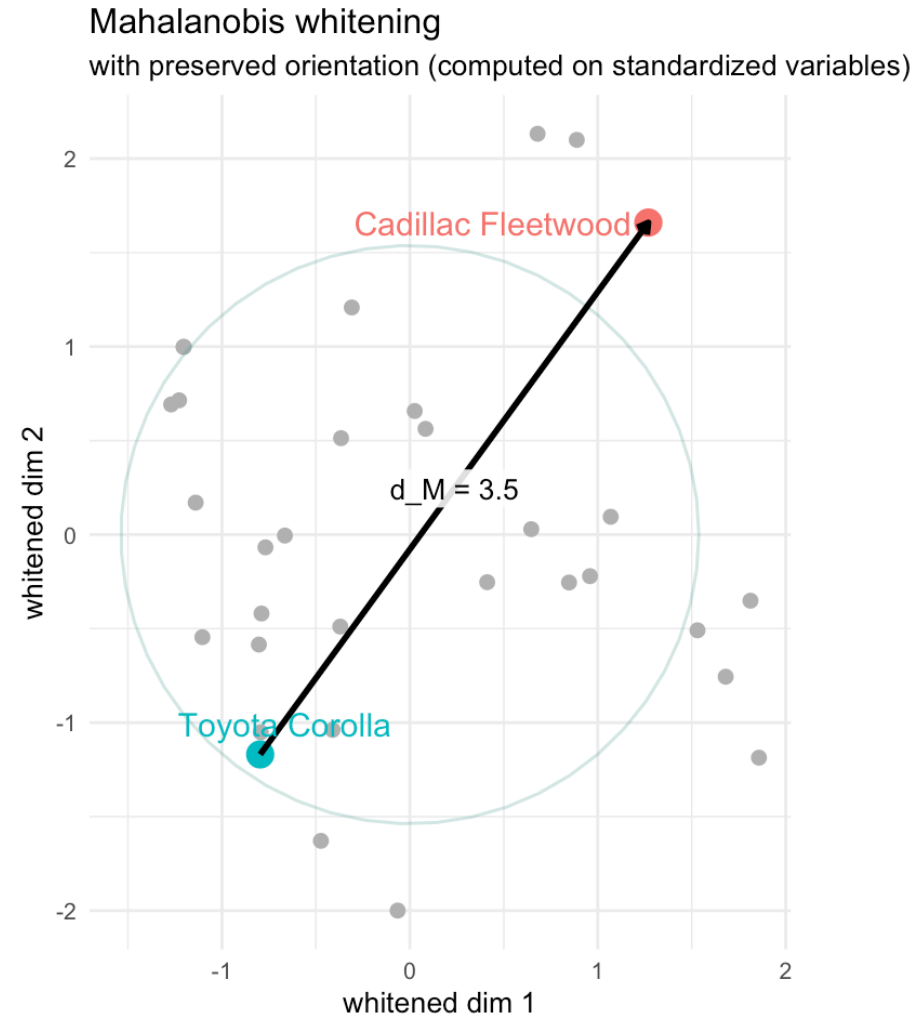
by variable differences: **independence-based**

The **Euclidean distance** → shared information is over-counted



accounting for inter-variable relations: **association-based**

The **Mahalanobis distance** → shared information is not over-counted



this is an **association-based** distance for continuous data

association-based distance

Association-based for continuous: Mahalanobis distance

Let $\mathbf{X}_{con}$ be $n \times Q_d$ a data matrix of n observations described by Q_d continuous variables, and let $\mathbf{S}$ the sample covariance matrix, the Mahalanobis distance matrix is

$$\mathbf{D}_{mah} = [\text{diag}(\mathbf{G}) \mathbf{1}_n^\top + \mathbf{1}_n \text{diag}(\mathbf{G})^\top - 2\mathbf{G}]^{\odot 1/2}$$

where

- $[\cdot]^{\odot 1/2}$ denotes the element-wise square root
- $\mathbf{G} = (\mathbf{C}\mathbf{X}_{con})\mathbf{S}^{-1}(\mathbf{C}\mathbf{X}_{con})^\top$ is the Mahalanobis Gram matrix
- $\mathbf{C} = \mathbf{I}_n - \frac{1}{n}\mathbf{1}_n\mathbf{1}_n^\top$ is the centering operator

association-based distance

Association-based for categorical: **total variation distance (TVD)** (Le & Ho, 2005)

The distance matrix $\mathbf{D}_{tvd}$ can be defined via the **delta framework**¹ upon properly defining the block-diagonal matrix Δ . Let $\mathbf{X}_{cat}$ be $n \times Q_c$ a data matrix of n observations described by Q_c categorical variables.

$$\mathbf{D} = \mathbf{Z}\Delta\mathbf{Z}^T = [\mathbf{Z}_1 \quad \dots \quad \mathbf{Z}_{Q_c}] \begin{bmatrix} \Delta_1 & & \\ & \ddots & \\ & & \Delta_{Q_c} \end{bmatrix} \begin{bmatrix} \mathbf{Z}_1^T \\ \vdots \\ \mathbf{Z}_{Q_c}^T \end{bmatrix}$$

- in the framework, setting Δ_j determines the categorical distance measure of choice (independent- or association-based)

association-based distance

Association-based for categorical: **total variation distance (TVD)** (Le & Ho, 2005) (2)

Consider the empirical **joint probability** distributions stored in the off-diagonal blocks of $\mathbf{P}$:

$$\mathbf{P} = \frac{1}{n} \begin{bmatrix} \mathbf{Z}_1^\top \mathbf{Z}_1 & \mathbf{Z}_1^\top \mathbf{Z}_2 & \cdots & \mathbf{Z}_1^\top \mathbf{Z}_{Q_c} \\ \vdots & \ddots & \vdots & \vdots \\ \mathbf{Z}_{Q_c}^\top \mathbf{Z}_1 & \mathbf{Z}_{Q_c}^\top \mathbf{Z}_2 & \cdots & \mathbf{Z}_{Q_c}^\top \mathbf{Z}_{Q_c} \end{bmatrix}.$$

The block matrix $\mathbf{R}$ refer to the **conditional probability** distributions for each variable j given each variable i ($i, j = 1, \dots, Q_c, i \neq j$), stored in the block matrix

$$\mathbf{R} = \mathbf{P}_z^{-1}(\mathbf{P} - \mathbf{P}_z).$$

where $\mathbf{P}_z = \mathbf{P} \odot \mathbf{I}_{Q^*}$, and $\mathbf{I}_{Q^*}$ is the $Q^* \times Q^*$ identity matrix.

association-based distance

Association-based for categorical: **total variation distance (TVD)**(Le & Ho, 2005) (3)

Let $\mathbf{r}_a^{ji}$ and $\mathbf{r}_b^{ji}$ be the rows of $\mathbf{R}_{ji}$, the (j, i) th off-diagonal block of $\mathbf{R}$.

The category dissimilarity between a and b for variable j based on the total variation distance (TVD) is defined as

$$\delta_{tvd}^j(a, b) = \sum_{i \neq j}^{Q_c} w_{ji} \Phi^{ji}(\mathbf{r}_a^{ji}, \mathbf{r}_b^{ji}) = \sum_{i \neq j}^{Q_c} w_{ji} \left[\frac{1}{2} \sum_{\ell=1}^{q_i} |\mathbf{r}_{a\ell}^{ji} - \mathbf{r}_{b\ell}^{ji}| \right],$$

where $w_{ji} = 1/(Q_c - 1)$ for equal weighting (can be user-defined).

TVD-based dissimilarity matrix is, therefore,

$$\mathbf{D}_{tvd} = \mathbf{Z} \Delta^{(tvd)} \mathbf{Z}^T.$$

association-based distance: a small example

Data

Consider two categorical variables:

- X_1 with categories A, B, C
- X_2 with categories u, v

```
# A tibble: 10 × 3
  id X1  X2
<int> <fct> <fct>
1     1  A   u
2     2  A   u
3     3  A   v
4     4  B   u
5     5  B   u
6     6  B   v
7     7  C   u
8     8  C   v
9     9  C   v
10    10  C   v
```

Indicator matrices

$$\mathbf{Z}_1 = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{pmatrix}, \quad \mathbf{Z}_2 = \begin{pmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 0 & 1 \end{pmatrix}.$$

association-based distance: from $\mathbf{Z}$ to $\mathbf{P}$

Let

$$\mathbf{Z} = [\mathbf{Z}_1, \mathbf{Z}_2].$$

The empirical co-occurrence matrix is

$$\mathbf{P} = \frac{1}{10} \mathbf{Z}^\top \mathbf{Z}.$$

For this example,

$$\mathbf{P} = \begin{pmatrix} 0.30 & 0 & 0 & 0.20 & 0.10 \\ 0 & 0.30 & 0 & 0.20 & 0.10 \\ 0 & 0 & 0.40 & 0.10 & 0.30 \\ 0.20 & 0.20 & 0.10 & 0.50 & 0 \\ 0.10 & 0.10 & 0.30 & 0 & 0.50 \end{pmatrix}.$$

diagonal blocks contain marginal information; off-diagonal blocks contain joint proportions

association-based distance: from $\mathbf{P}$ to $\mathbf{R}$

The diagonal part of $\mathbf{P}$ is

$$\mathbf{P}_z = \mathbf{P} \odot \mathbf{I}_{Q^*} = \text{diag}(0.30, 0.30, 0.40, 0.50, 0.50).$$

The block matrix of conditional profiles is

$$\mathbf{R} = \mathbf{P}_z^{-1}(\mathbf{P} - \mathbf{P}_z).$$

For this example,

$$\mathbf{R} = \begin{pmatrix} 0 & 0 & 0 & 0.67 & 0.33 \\ 0 & 0 & 0 & 0.67 & 0.33 \\ 0 & 0 & 0 & 0.25 & 0.75 \\ 0.40 & 0.40 & 0.20 & 0 & 0 \\ 0.20 & 0.20 & 0.60 & 0 & 0 \end{pmatrix}.$$

each off-diagonal block contains conditional profiles across variables

association-based distance: reading $\mathbf{R}_{12}$

For the categories of X_1 , the relevant block is

$$\mathbf{R}_{12} = \begin{pmatrix} 0.67 & 0.33 \\ 0.67 & 0.33 \\ 0.25 & 0.75 \end{pmatrix}.$$

Interpretation

Rows of $\mathbf{R}_{12}$ describe the distribution of X_2 within each category of X_1 :

- $P(X_2 = u \mid X_1 = A) = 0.67, P(X_2 = v \mid X_1 = A) = 0.33$
- $P(X_2 = u \mid X_1 = B) = 0.67, P(X_2 = v \mid X_1 = B) = 0.33$
- $P(X_2 = u \mid X_1 = C) = 0.25, P(X_2 = v \mid X_1 = C) = 0.75$

categories are compared through their association profiles

association-based distance: from $\mathbf{R}$ to $\Delta_1^{(tvd)}$

Compare the rows of $\mathbf{R}_{12}$ using TVD.

$$\delta_{tvd}^1(A, B) = \frac{1}{2}(|0.67 - 0.67| + |0.33 - 0.33|) = 0.$$

$$\delta_{tvd}^1(A, C) = \frac{1}{2}(|0.67 - 0.25| + |0.33 - 0.75|) = 0.42.$$

$$\delta_1^{tvd}(B, C) = 0.42.$$

Therefore,

$$\Delta_1^{(tvd)} = \begin{pmatrix} 0 & 0 & 0.42 \\ 0 & 0 & 0.42 \\ 0.42 & 0.42 & 0 \end{pmatrix}.$$

A and B are close because they have the same profile with respect to X_2

association-based distance: reading $\mathbf{R}_{21}$

For the categories of X_2 , the relevant block is

$$\mathbf{R}_{21} = \begin{pmatrix} 0.40 & 0.40 & 0.20 \\ 0.20 & 0.20 & 0.60 \end{pmatrix}.$$

Interpretation

Rows of $\mathbf{R}_{21}$ describe the distribution of X_1 within each category of X_2 :

- $P(X_1 = A \mid X_2 = u) = 0.40, P(X_1 = B \mid X_2 = u) = 0.40, P(X_1 = C \mid X_2 = u) = 0.20$
- $P(X_1 = A \mid X_2 = v) = 0.20, P(X_1 = B \mid X_2 = v) = 0.20, P(X_1 = C \mid X_2 = v) = 0.60$

categories of X_2 are compared through their association profiles with respect to X_1

association-based distance: from $\mathbf{R}_{21}$ to $\Delta_2^{(tvd)}$

Compare the rows of $\mathbf{R}_{21}$ using TVD.

$$\delta_1^{tvd}(u, v) = \frac{1}{2}(|0.40 - 0.20| + |0.40 - 0.20| + |0.20 - 0.60|) = 0.40.$$

Therefore,

$$\Delta_2^{(tvd)} = \begin{pmatrix} 0 & 0.40 \\ 0.40 & 0 \end{pmatrix}.$$

u and v are different because they imply different profiles over X_1

association-based distance: from Δ to $\mathbf{D}$

We collect the category dissimilarity matrices in a block-diagonal matrix:

$$\Delta^{(tvd)} = \begin{pmatrix} \Delta_1^{(tvd)} & 0 \\ 0 & \Delta_2^{(tvd)} \end{pmatrix}.$$

The observation-level categorical distance matrix is then

$$\mathbf{D}_{tvd} = \mathbf{Z}\Delta^{(tvd)}\mathbf{Z}^\top = [\mathbf{Z}_1 \quad \mathbf{Z}_2] \begin{pmatrix} \Delta_1^{(tvd)} & 0 \\ 0 & \Delta_2^{(tvd)} \end{pmatrix} \begin{bmatrix} \mathbf{Z}_1^\top \\ \mathbf{Z}_2^\top \end{bmatrix}.$$

Equivalently,

$$\mathbf{D}_{tvd} = \mathbf{Z}_1\Delta_1^{(tvd)}\mathbf{Z}_1^\top + \mathbf{Z}_2\Delta_2^{(tvd)}\mathbf{Z}_2^\top.$$

category-level dissimilarities are translated into observation-level distances

No one-size-fits-all distance

The need for diagnostics

From distances to data representation

Different distance definitions induce different **distance-based representations** of the same data.

Same data, different representation

Changing the distance changes the **global dissimilarity structure** on which downstream learning methods rely.

Leave-one-variable-out diagnostics

How can we measure the contribution of each variable to this structure?

- compare the dissimilarity matrix computed with and without the variable in question

LOVO-based benchmark: evaluated distances

The benchmark compares distance definitions that differ in how they treat **scale**, **type**, **additivity**, and **association**.

Additive distances

- **gower** : classical Gower dissimilarity
- **mod_gower** : modified Gower coefficients ([Liu et al., 2024](#))
- **hl_add** : additive version of Hennig–Liao scaling ([C. Hennig & Liao, 2013](#))
- **u_ind** : unbiased independence-based distance
- **u_dep** : unbiased association-based distance
- **u_mix** : unbiased Manhattan and TVD

Non-additive distances

- **naive** : Euclidean distance on scaled numerical variables and one-hot-encoded
- **hl** : Hennig–Liao scaling with Euclidean distance
- **gudmm** : generalized multi-aspect distance metric for mixed-type data ([Mousavi & Sehhati, 2023](#))
- **dkps** : distance using kernel product similarity ([Ghashti & Thompson, 2025](#))

LOVO-based diagnostics: what is evaluated?

For each distance and each variable X_j , we compare the full-data representation $\mathbf{D}$ with the representation obtained after removing X_j , that is $\mathbf{D}_{-j}$.

1. Distance level

Numeric comparison between $\mathbf{D}$ and $\mathbf{D}_{-j}$.

- **mean absolute difference** between distance matrices.

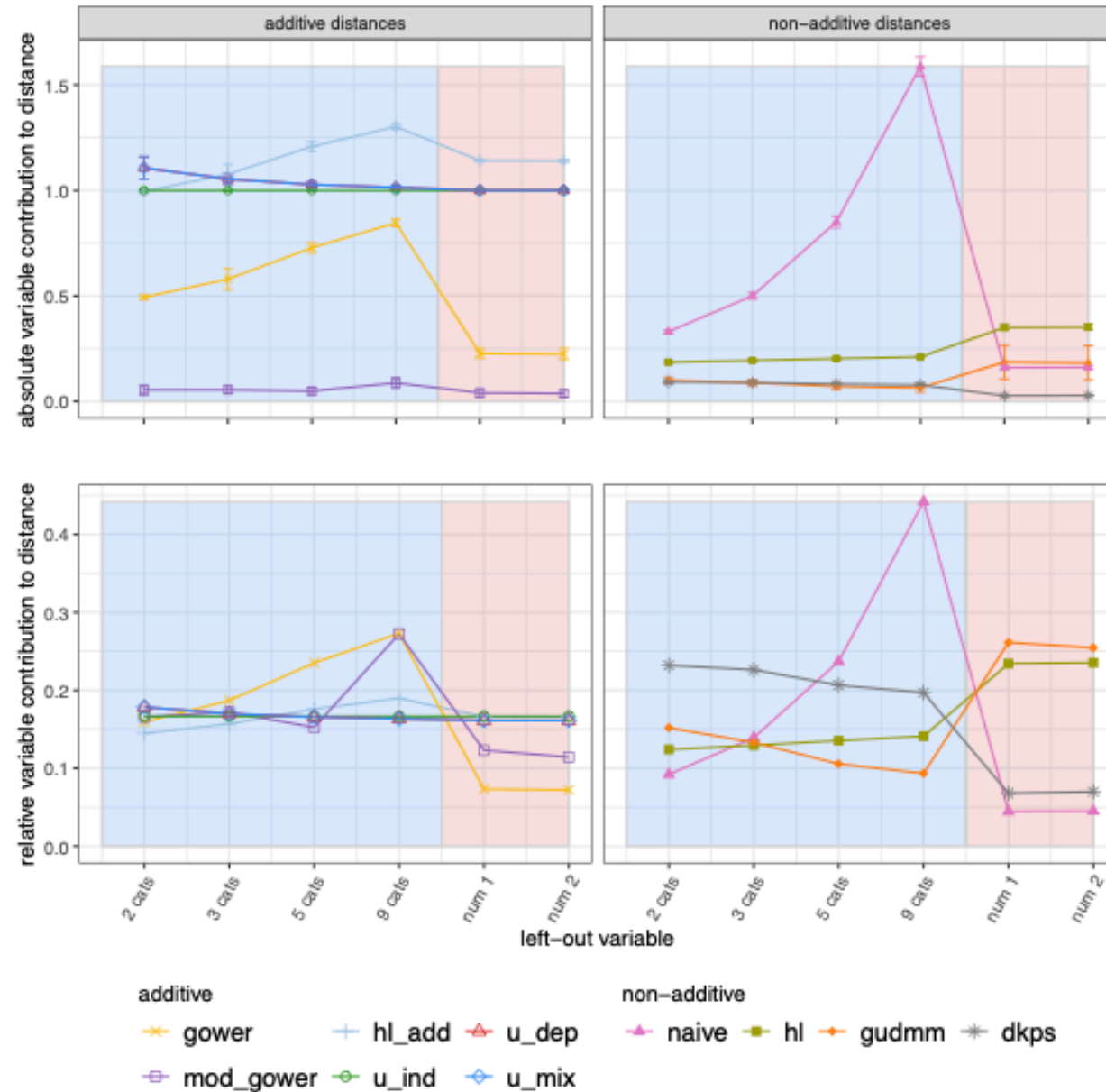
2. MDS level

Compute MDS from $\mathbf{D}$ and from $\mathbf{D}_{-j}$, then compare the resulting configurations.

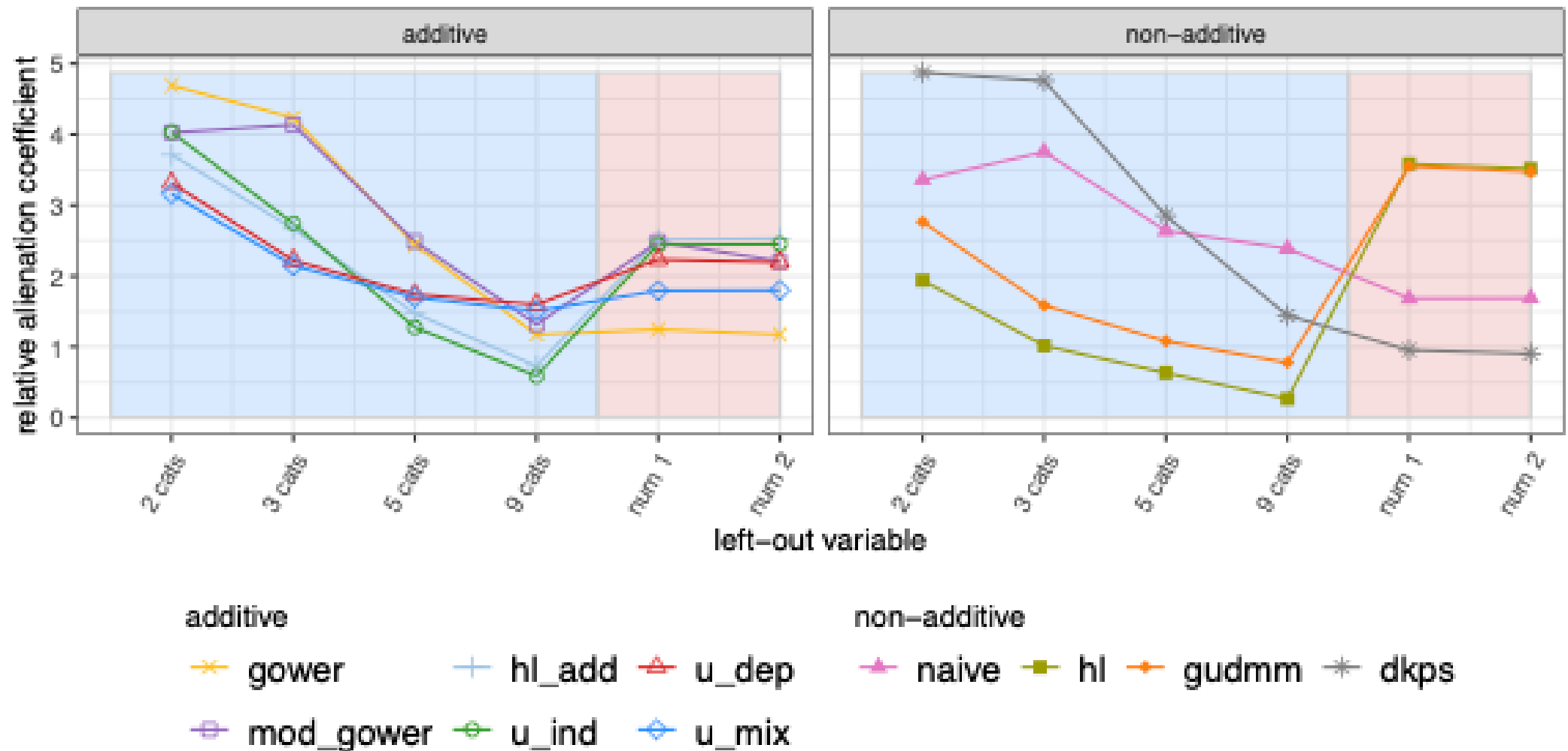
- **alienation coefficient** between MDS representations.

LOVO diagnostics assess how each variable contributes to the dissimilarity structure

LOVO diagnostics: distance-level effect



LOVO diagnostics: MDS-level effect



- commensurability balances expected distance contributions, not necessarily the role of variables in every downstream representation

From diagnostics to downstream learning

LOVO diagnostics show how variables affect the distance matrix and the MDS representation.

But we also want to know whether distance biases affect a downstream learning task.

Clustering experiment

Use each distance matrix as input to PAM and evaluate how well the resulting partition recovers the known cluster structure.

Clustering experiment

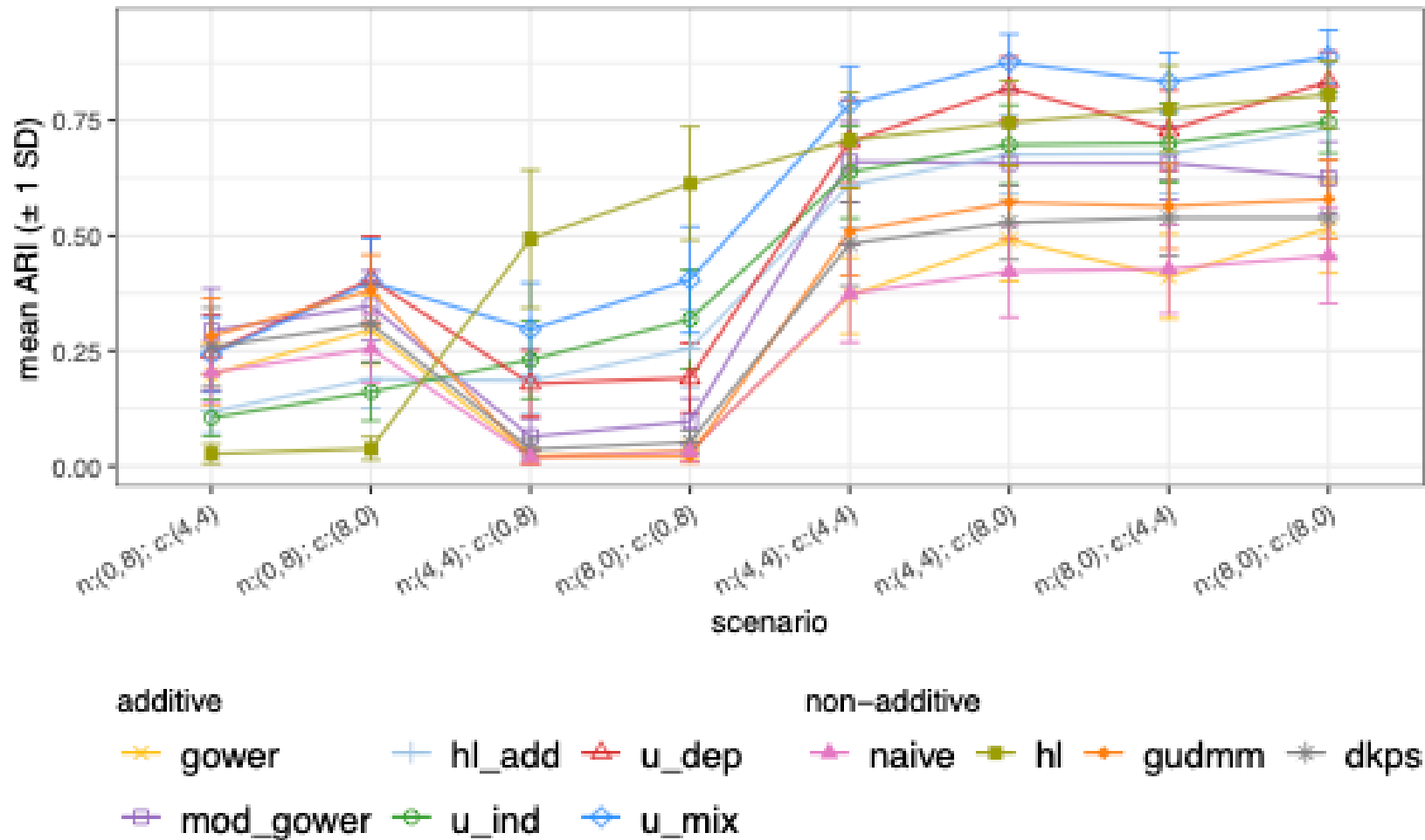
Data generation

- (n = 200) observations from (4) equal-sized clusters
- data generated with `genRandomClust`
- each dataset contains (8) numerical and (8) categorical variables
- categorical variables are obtained by discretizing numerical variables into (9) categories
- scenarios vary the number of signal and noise variables within each type
- (100) datasets are generated for each scenario

Evaluation

For each mixed-data distance, PAM is applied to the dissimilarity matrix with (K = 4). Recovery of the true cluster labels is measured using the adjusted Rand index.

PAM-based clustering results



- `hl` performs well when categorical variables are noise, but poorly when numerical variables are noise
- `gower` tends to show the opposite pattern
- `u_mix` and `u_dep` are comparatively stable in the mixed signal/noise scenarios

4. Interaction-aware extensions

Cross-type interactions

Association-aware distances account for relations **within** variable blocks:

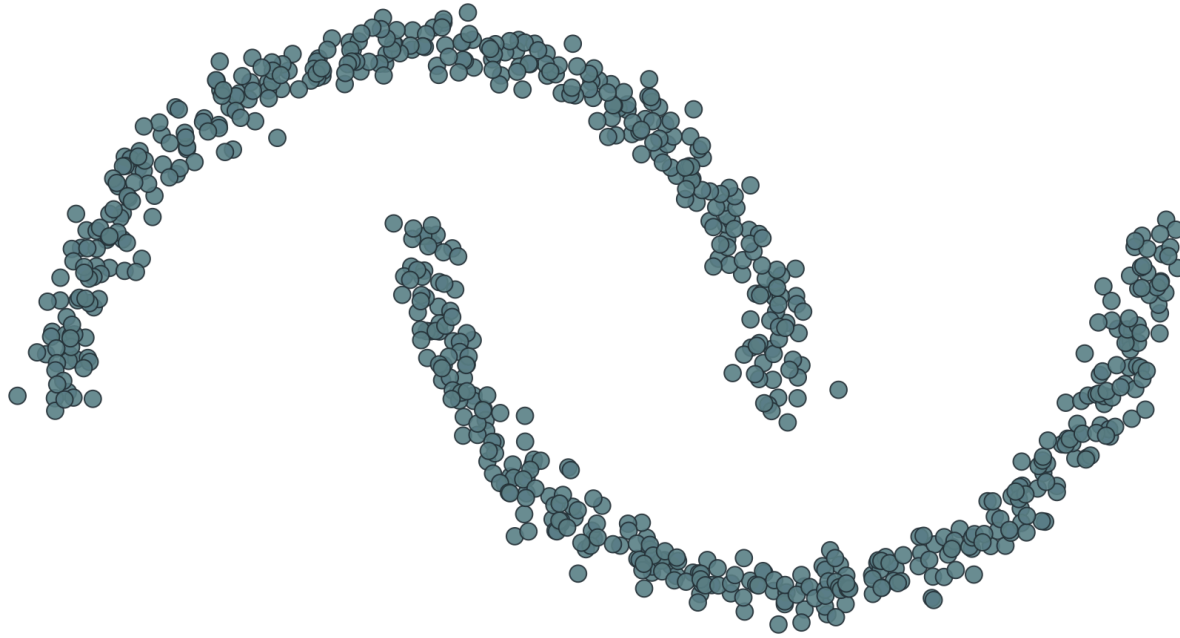
- continuous–continuous relations;
- categorical–categorical relations.

Cross-type structure

In mixed data, categorical differences may be meaningful because they are reflected in the continuous variables.

the next step is to make distances **interaction-aware**

The numerical geometry suggests two groups



2

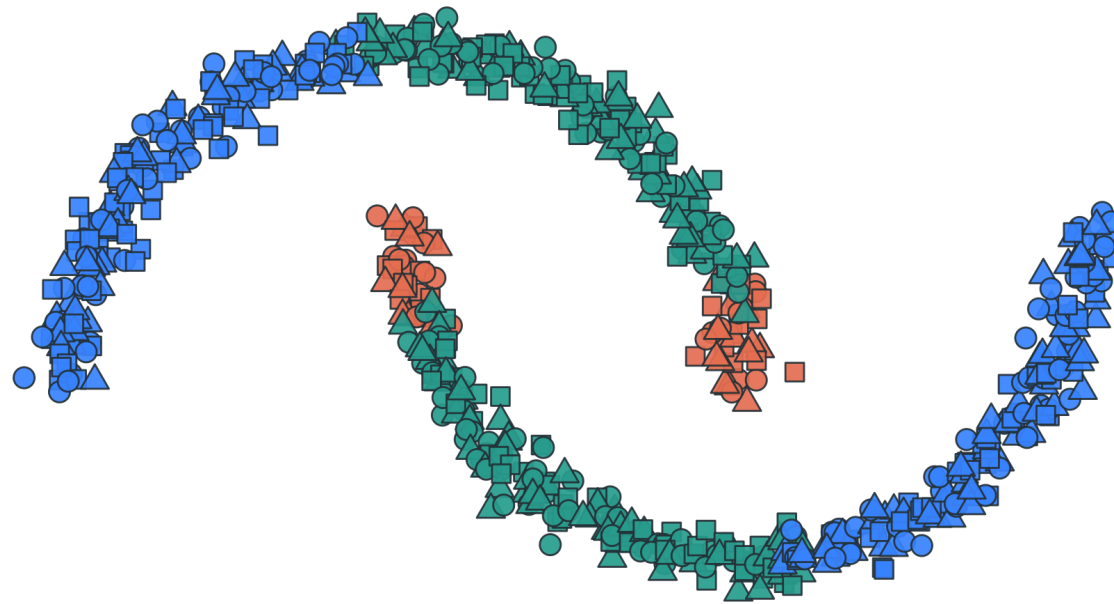
connected shapes

A numerical distance can recover the **upper (U)** and **lower (L)** moon.

Geometry alone

There is no preferred subdivision within either moon.

One factor is structured; the other is deliberately neutral



Signal band ● A ● B ● C Nuisance ● N1 ▲ N2 ■ N3

Signal band

- colour: A, B, C
- unequal arc lengths: 10%, 45%, 45%
- repeated on both moons

Noise factor

- shape: N_1, N_2, N_3
- balanced within every band
- spatially random

only the signal factor has a systematic relationship with the numerical geometry

The target is the moon **AND** the signal category

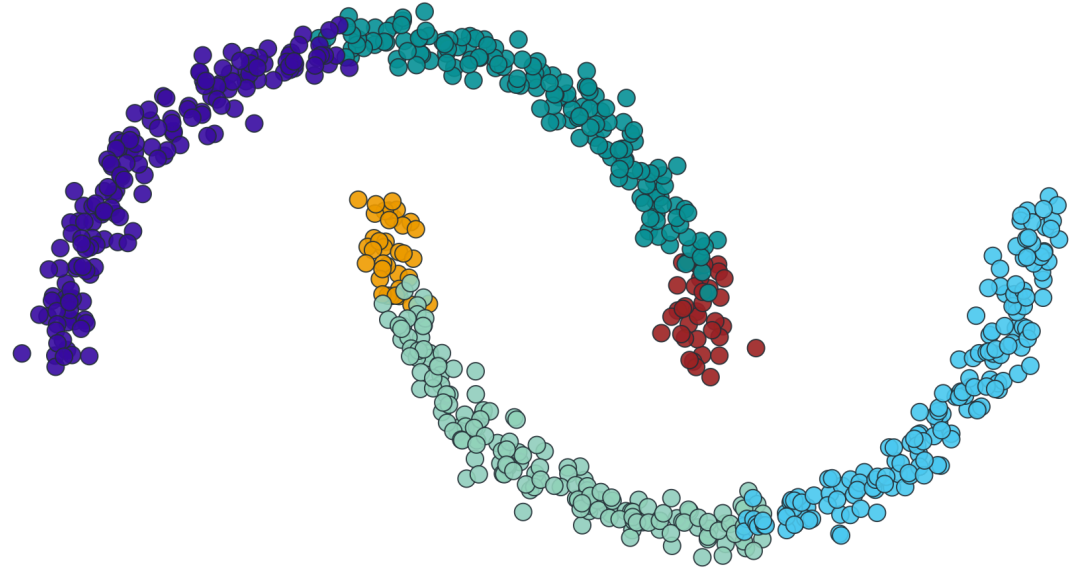
$$\text{cluster} = \text{moon} \times \text{band}$$

The desired partition contains six groups:

$UA, UB, UC,$
 $LA, LB, LC.$

The six-way intersection

Moon alone gives two groups. Band alone gives three.



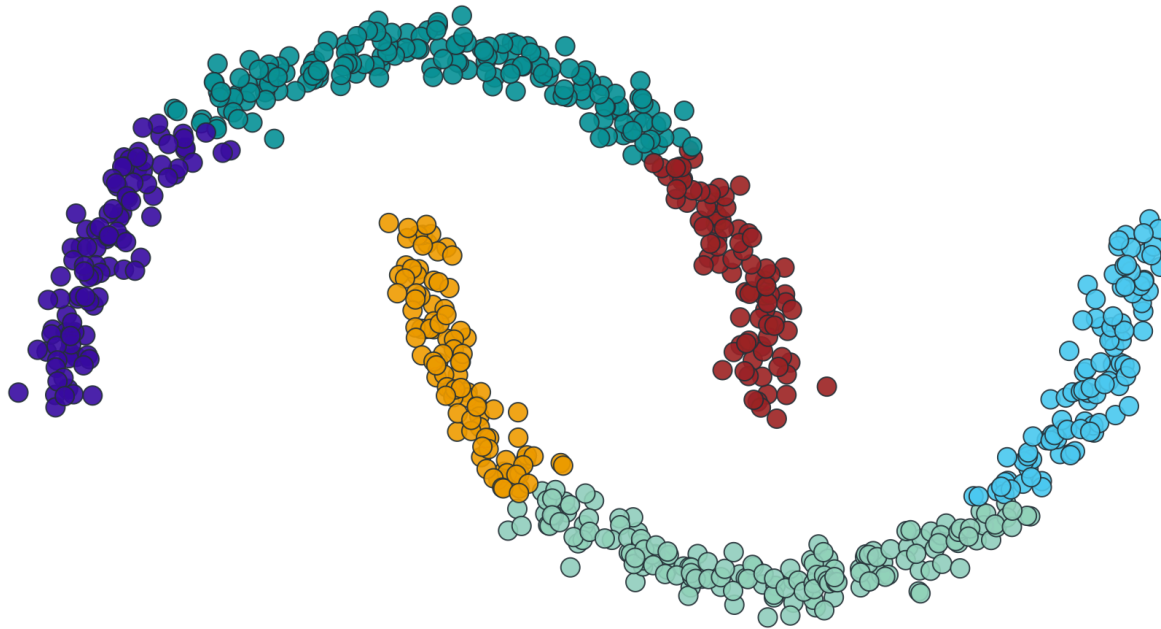
Without interaction, the graph follows geometry

ARI = 0.490

Spectral clustering is asked for $K = 6$, but the graph does not know where A , B , and C change.

The wrong boundaries

The moons are subdivided according to numerical geometry—not the signal factor.



With interaction, the graph recovers the six intersections

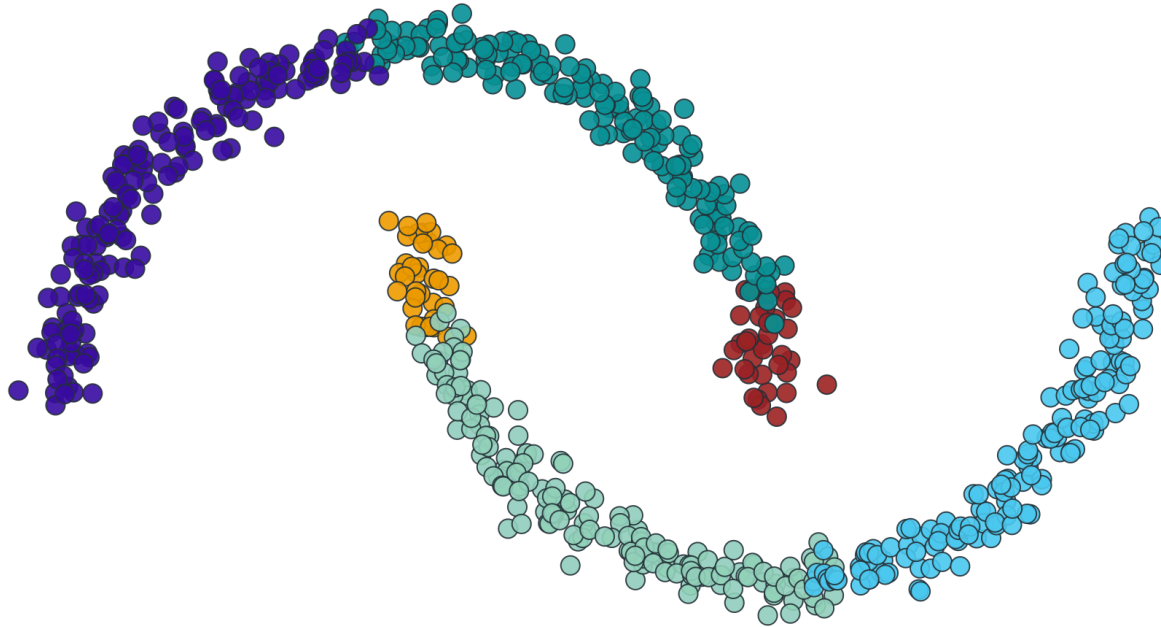
ARI = 1.000

The numerical distance preserves the two moons.

Δ_{int} adds barriers at the $A/B/C$ boundaries.

Exact recovery

The graph now encodes the required **moon AND category** structure.



How to measure interactions¹

Define Δ^{int} to account for continuous–categorical interactions and use it to augment Δ^{tvd} .

The mixed dissimilarity becomes

$$\mathbf{D}_{mix}^{(int)} = \mathbf{D}_{mah} + \mathbf{D}_{cat}^{(int)}.$$

where

$$\mathbf{D}_{cat}^{(int)} = \mathbf{Z}\tilde{\Delta}\mathbf{Z}^\top$$

and

$$\tilde{\Delta} = (1 - \alpha)\Delta^{tvd} + \alpha\Delta^{int}, \quad \alpha = \frac{1}{Q_c}.$$

What is Δ^{int} ?

The entry $\delta_{int}^j(a, b)$ measures how much the continuous variables help discriminate between observations choosing category a and those choosing category b for categorical variable j .

Category-pair classification problem

For each pair (a, b) :

- use the continuous variables as predictors;
- classify observations belonging to categories a and b ;
- use a nearest-neighbour rule in the continuous space.

Computing Δ_j^{int}

For each categorical variable j and each category pair (a, b) :

1. use $\mathbf{D}_{mah}$ to identify neighbours among observations belonging to a or b ;
2. consider a proportion of neighbours, say $\pi_{nn} = 0.1$;
3. classify observations using a prior-corrected decision rule;
4. compute balanced accuracy.

$$\text{BAcc}^j(a, b) = \frac{1}{2} \left[\text{TPR}_a^j(a, b) + \text{TPR}_b^j(a, b) \right].$$

Map predictive performance onto a non-negative separability scale:

$$\delta_{int}^j(a, b) = \max \{0, 2 \text{BAcc}^j(a, b) - 1\} \in [0, 1].$$

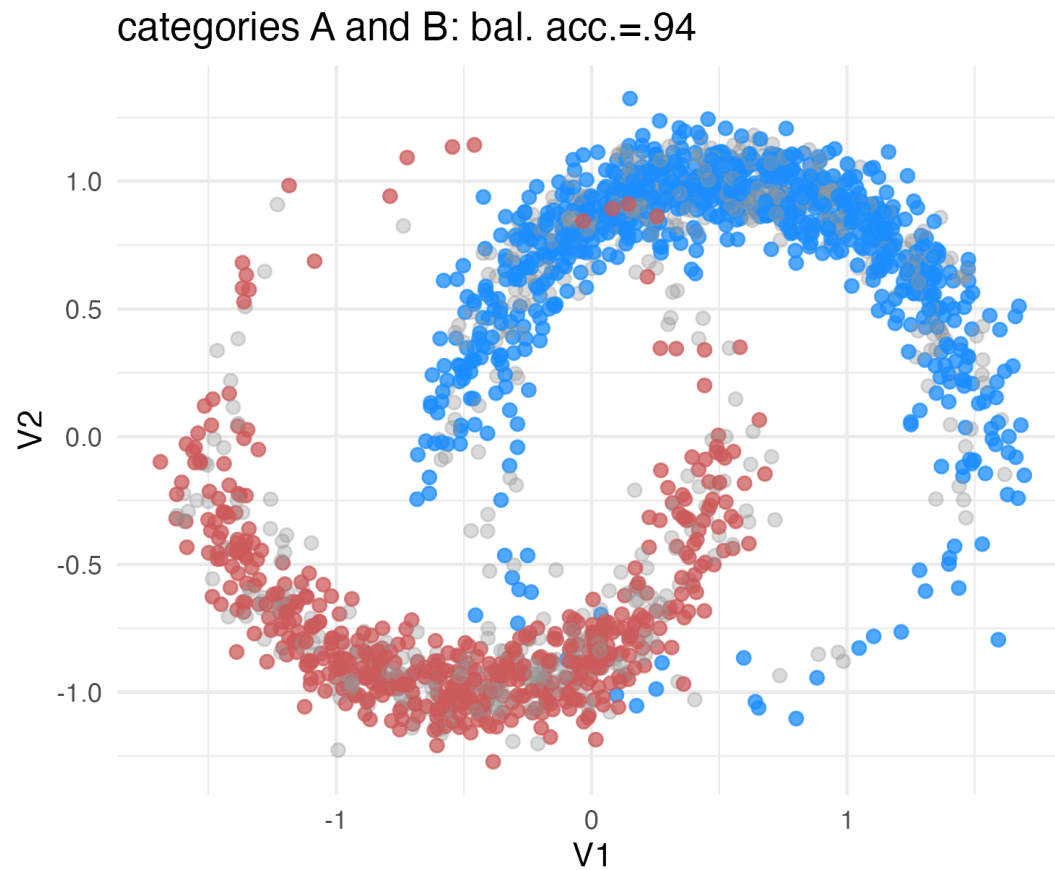
0 means chance-level or worse; 1 means perfect separation

Computing Δ_j^{int}

For categorical variable j with q_j categories, compute $\frac{q_j(q_j-1)}{2}$ category-pair quantities.

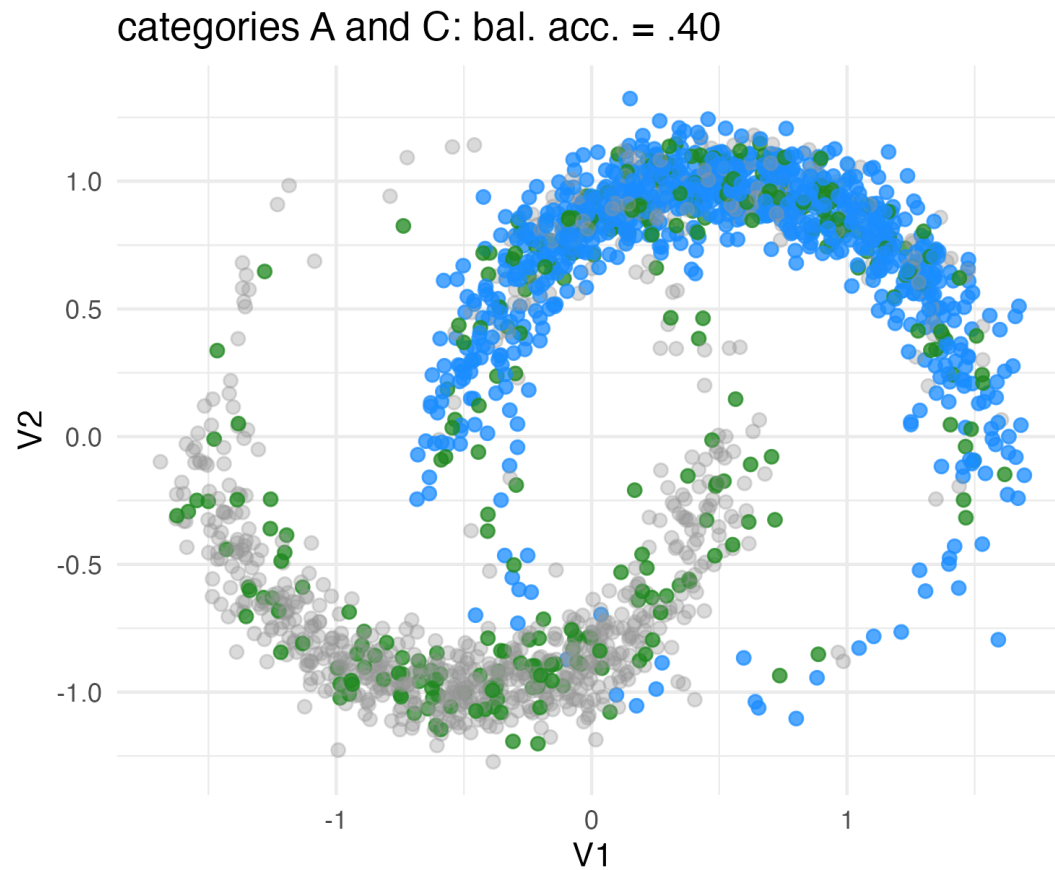
$$\Delta_{int} = \begin{pmatrix} 0 & \cdot & \cdot & \cdot \\ \cdot & 0 & \cdot & \cdot \\ \cdot & \cdot & 0 & \cdot \\ \cdot & \cdot & \cdot & 0 \end{pmatrix}$$

Computing Δ_j^{int}



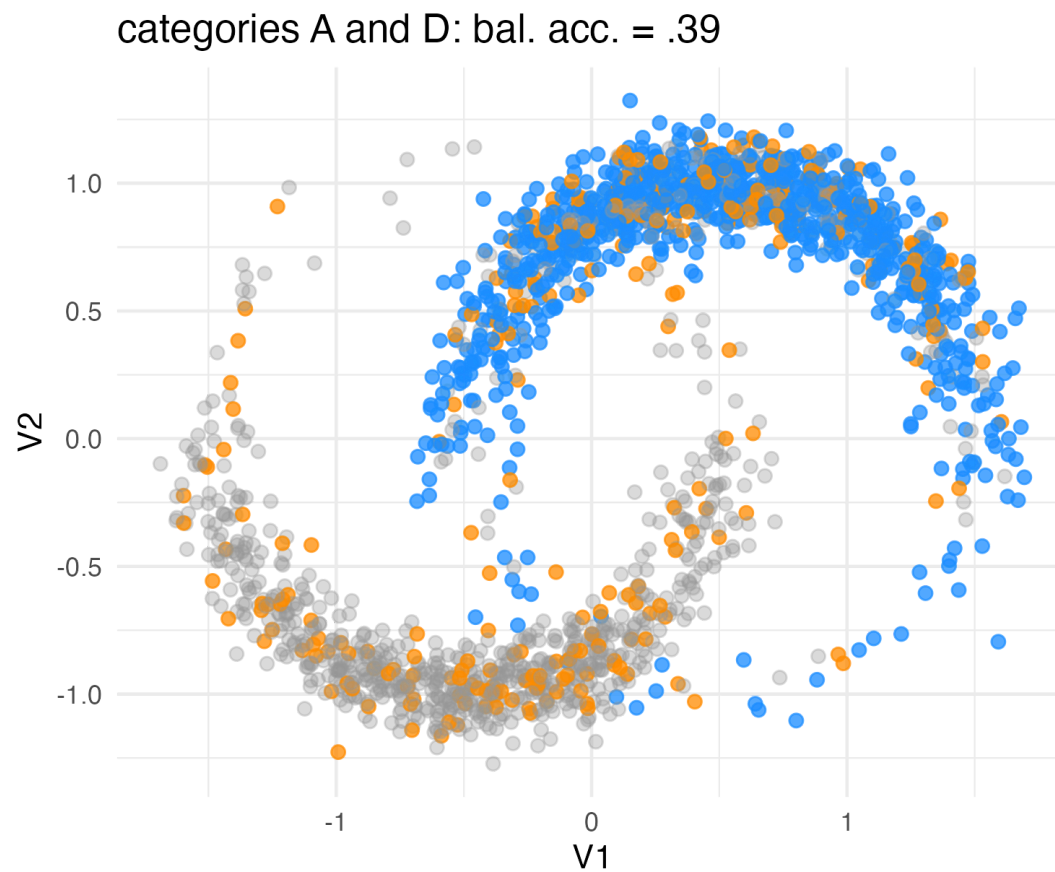
$$\Delta_{int} = \begin{pmatrix} 0 & 0.94 & \cdot & \cdot \\ 0.94 & 0 & \cdot & \cdot \\ \cdot & \cdot & 0 & \cdot \\ \cdot & \cdot & \cdot & 0 \end{pmatrix}$$

Computing Δ_j^{int}



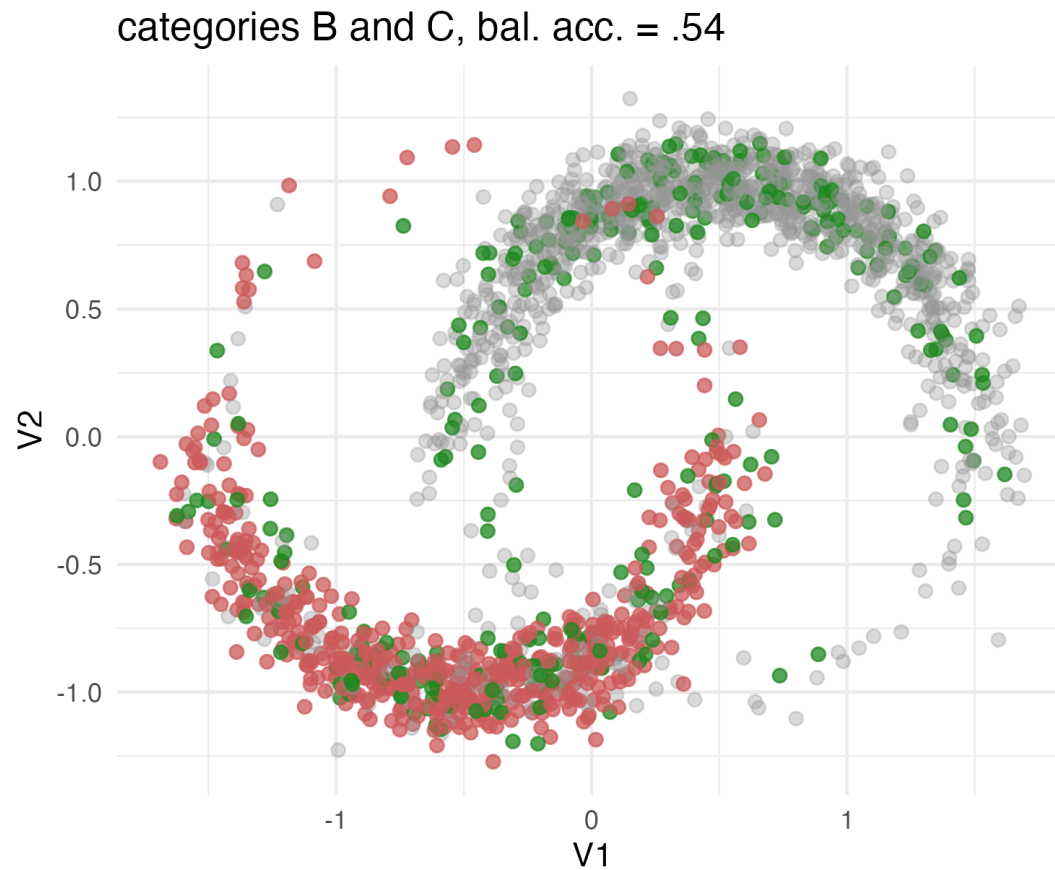
$$\Delta_{int} = \begin{pmatrix} 0 & 0.94 & \textcolor{red}{0.40} & \cdot \\ 0.94 & 0 & \cdot & \cdot \\ \textcolor{red}{0.40} & \cdot & 0 & \cdot \\ \cdot & \cdot & \cdot & 0 \end{pmatrix}$$

Computing Δ_j^{int}



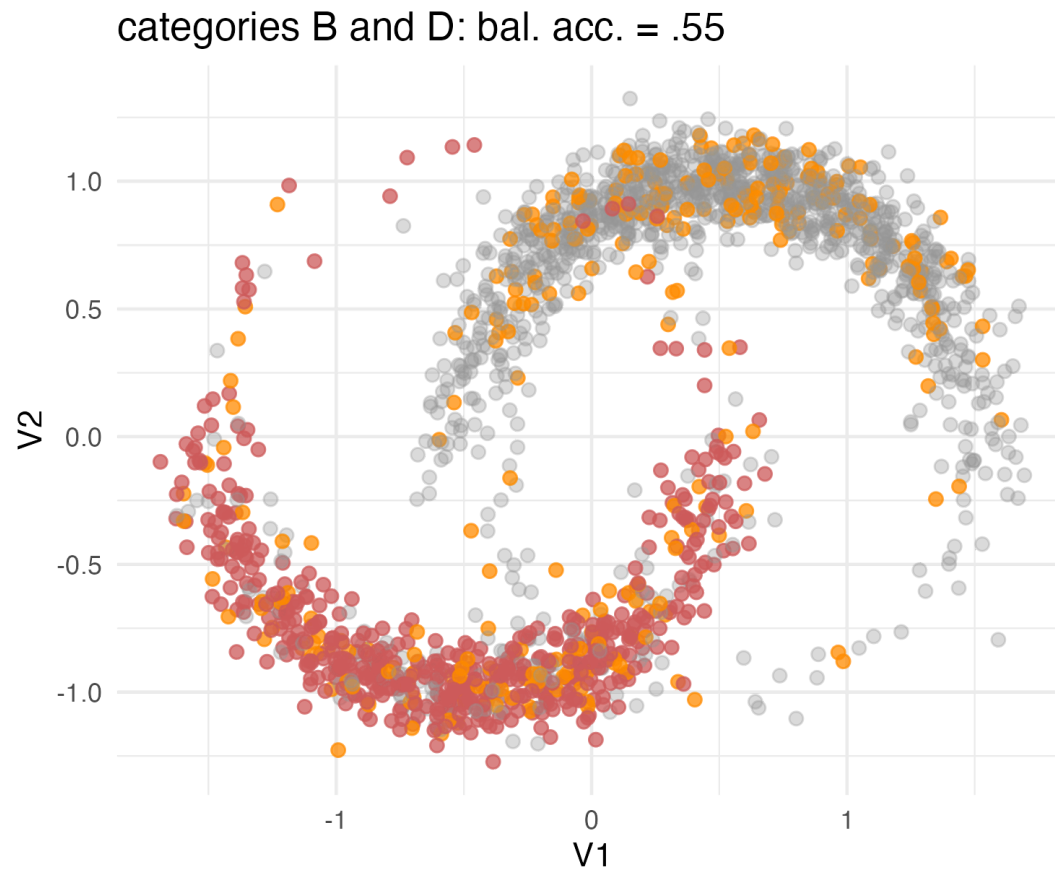
$$\Delta_{int} = \begin{pmatrix} 0 & 0.94 & 0.40 & \textcolor{red}{0.39} \\ 0.94 & 0 & \cdot & \cdot \\ 0.40 & \cdot & 0 & \cdot \\ \textcolor{red}{0.39} & \cdot & \cdot & 0 \end{pmatrix}$$

Computing Δ_j^{int}



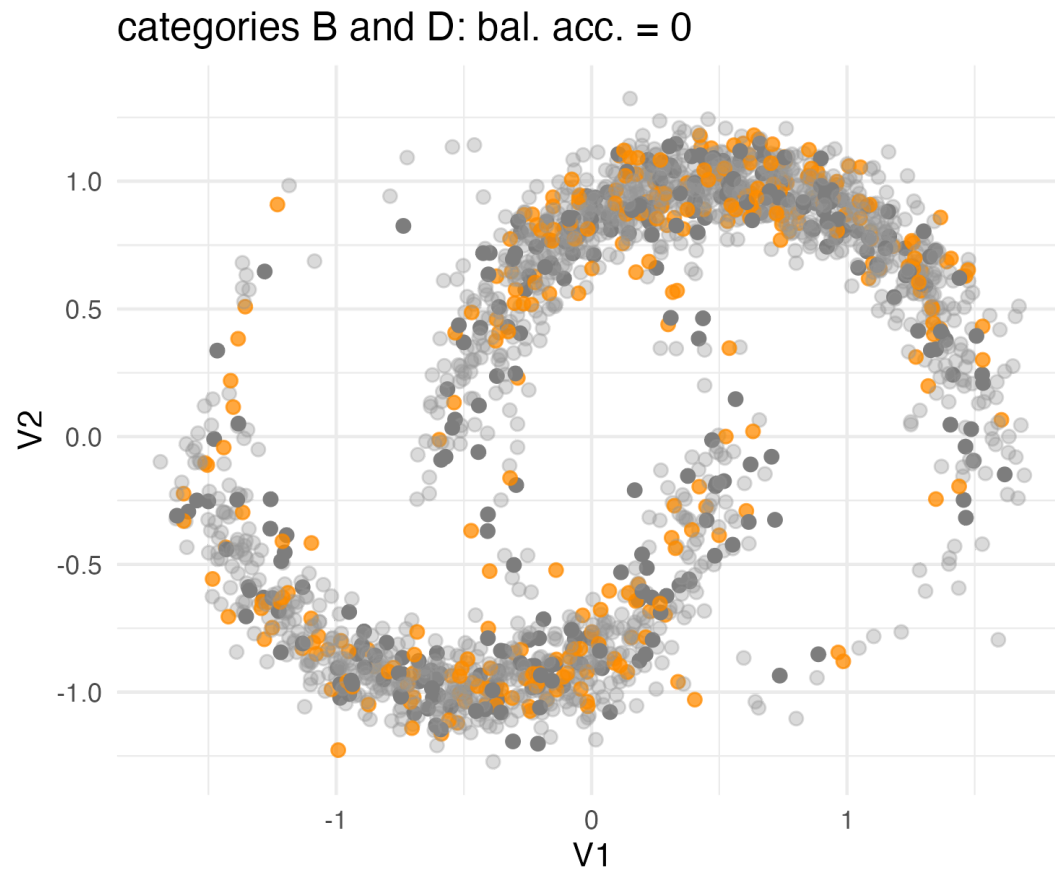
$$\Delta_{int} = \begin{pmatrix} 0 & 0.94 & 0.40 & 0.39 \\ 0.94 & 0 & 0.54 & . \\ 0.40 & 0.54 & 0 & . \\ 0.39 & . & . & 0 \end{pmatrix}$$

Computing Δ_j^{int}



$$\Delta_{int} = \begin{pmatrix} 0 & 0.94 & 0.40 & 0.39 \\ 0.94 & 0 & 0.54 & 0.55 \\ 0.40 & 0.54 & 0 & . \\ 0.39 & 0.55 & . & 0 \end{pmatrix}$$

Computing Δ_j^{int}



$$\Delta_{int} = \begin{pmatrix} 0 & 0.94 & 0.40 & 0.39 \\ 0.94 & 0 & 0.54 & 0.55 \\ 0.40 & 0.54 & 0 & 0 \\ 0.39 & 0.55 & 0 & 0 \end{pmatrix}$$

summarize category-pair separability in the continuous space

Just one-way interaction?

Factorization motivates the direction

For $\mathbf{x}_i = (\mathbf{x}_{i_n}, \mathbf{x}_{i_c})$,

$$f(\mathbf{x}_{i_n}, \mathbf{x}_{i_c}) = f(\mathbf{x}_{i_n})f(\mathbf{x}_{i_c} \mid \mathbf{x}_{i_n}).$$

We therefore use the numerical block as the reference representation for categorical distinctions.

Dissimilarity and a reference density

For a non-negative dissimilarity to a prototype, when the exponential kernel is normalisable,

$$f(\mathbf{x}; \mathbf{c}, \mathbf{S}) = g(\mathbf{c}, \mathbf{S}, \eta) \exp\{-\eta d_{\mathbf{S}}(\mathbf{x}, \mathbf{c})\}.$$

Since $M = f(\mathbf{c}; \mathbf{c}, \mathbf{S}) = g(\mathbf{c}, \mathbf{S}, \eta)$,

$$\begin{aligned}\bar{K}(\mathbf{x}, \mathbf{c}) &= \frac{f(\mathbf{x}; \mathbf{c}, \mathbf{S})}{M} = \exp\{-\eta d_{\mathbf{S}}(\mathbf{x}, \mathbf{c})\}, \\ d_{\mathbf{S}}(\mathbf{x}, \mathbf{c}) &= -\frac{1}{\eta} \log \bar{K}(\mathbf{x}, \mathbf{c}).\end{aligned}$$

Pairwise consequence

Take another observation as the reference prototype, $\mathbf{c} \leftarrow \mathbf{x}_{i'}$, and set $\eta = 1$:

$$\begin{aligned}\bar{K}_{\text{mix}}(i, i') &= \bar{K}_{\text{n}}(i, i') \bar{K}_{\text{c}|n}(i, i') \\ -\log \bar{K}_{\text{mix}}(i, i') &= d_{\text{n}}(i, i') + d_{\text{c}|n}(i, i').\end{aligned}$$

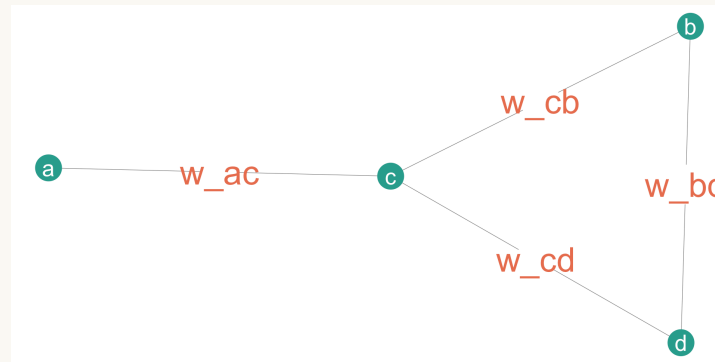
Distance-to-prototype construction: (Ben-Israel & Iyigun, 2008; Christian Hennig, Viroli, & Anderlucci, 2019). Full mixed-data derivation: (Iodice D'Enza et al., 2026).

downstream analysis

Spectral clustering: a graph partitioning problem

Graph representation

A graph representation of the data matrix $\mathbf{X}$: the aim is to cut it into K groups, or clusters.



The **affinity matrix $\mathbf{A}$**

The elements w_{ij} of $\mathbf{A}$ are high when observations i and j are likely to belong to the same group, and low otherwise.

Spectral clustering: making the graph easy to cut

An approximate solution to the graph partitioning problem (A. Ng, Jordan, & Weiss, 2001):

From distances to affinities

Start from the pairwise distance matrix $\mathbf{D}$ and build the affinity matrix

$$\mathbf{A} = \exp \left(-\frac{\mathbf{D}^2}{2\sigma^2} \right), \quad a_{ii} = 0.$$

The parameter σ controls the neighbourhood scale.

Normalized graph Laplacian

$$\mathbf{L} = \mathbf{D}_r^{-1/2} \mathbf{A} \mathbf{D}_r^{-1/2} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^\top,$$

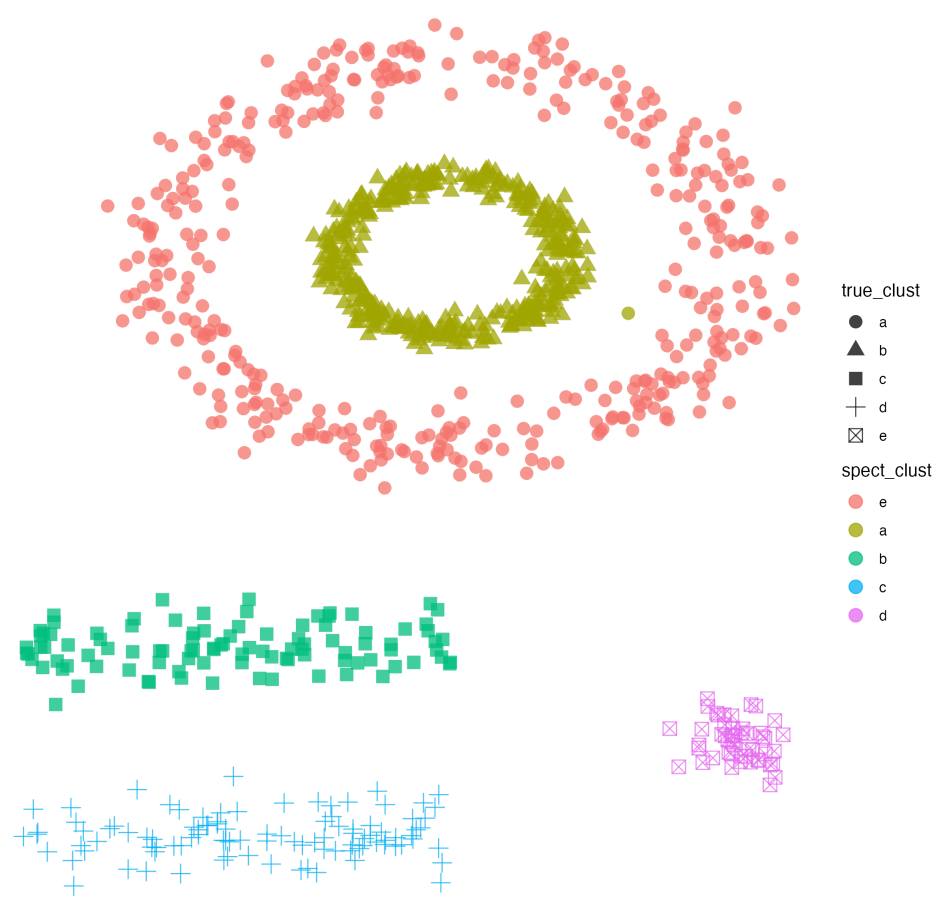
where $\mathbf{D}_r = \text{diag}(\mathbf{r})$, $\mathbf{r} = \mathbf{A}\mathbf{1}$, $\mathbf{1}$ is an I -dimensional vector of ones.

Spectral embedding

apply K -means to the rows of $\tilde{\mathbf{Q}}$, containing the first K eigenvectors of $\mathbf{L}$.

Why spectral clustering here?

Interaction-aware distances can encode local connectivity and non-convex structure.



Synthetic experiments

Simulation 1: associated mixed variables

General design

- two sample sizes: $I = 1000$ and $I = 2000$
- number of clusters fixed to $K = 3$
- 50 datasets generated for each configuration
- spectral clustering applied to each dissimilarity matrix
- performance measured by adjusted Rand index

Variable composition

Four scenarios were considered:

1. $Q_n = 15, Q_c = 15$
2. $Q_n = 10, Q_c = 20$
3. $Q_n = 20, Q_c = 10$
4. $Q_n = 10, Q_c = 10$ plus noise

Aim

The simulation creates mixed data with:

- correlated numerical variables;
- associated categorical variables;
- categorical variables depending on numerical variables;
- an additional noise scenario.

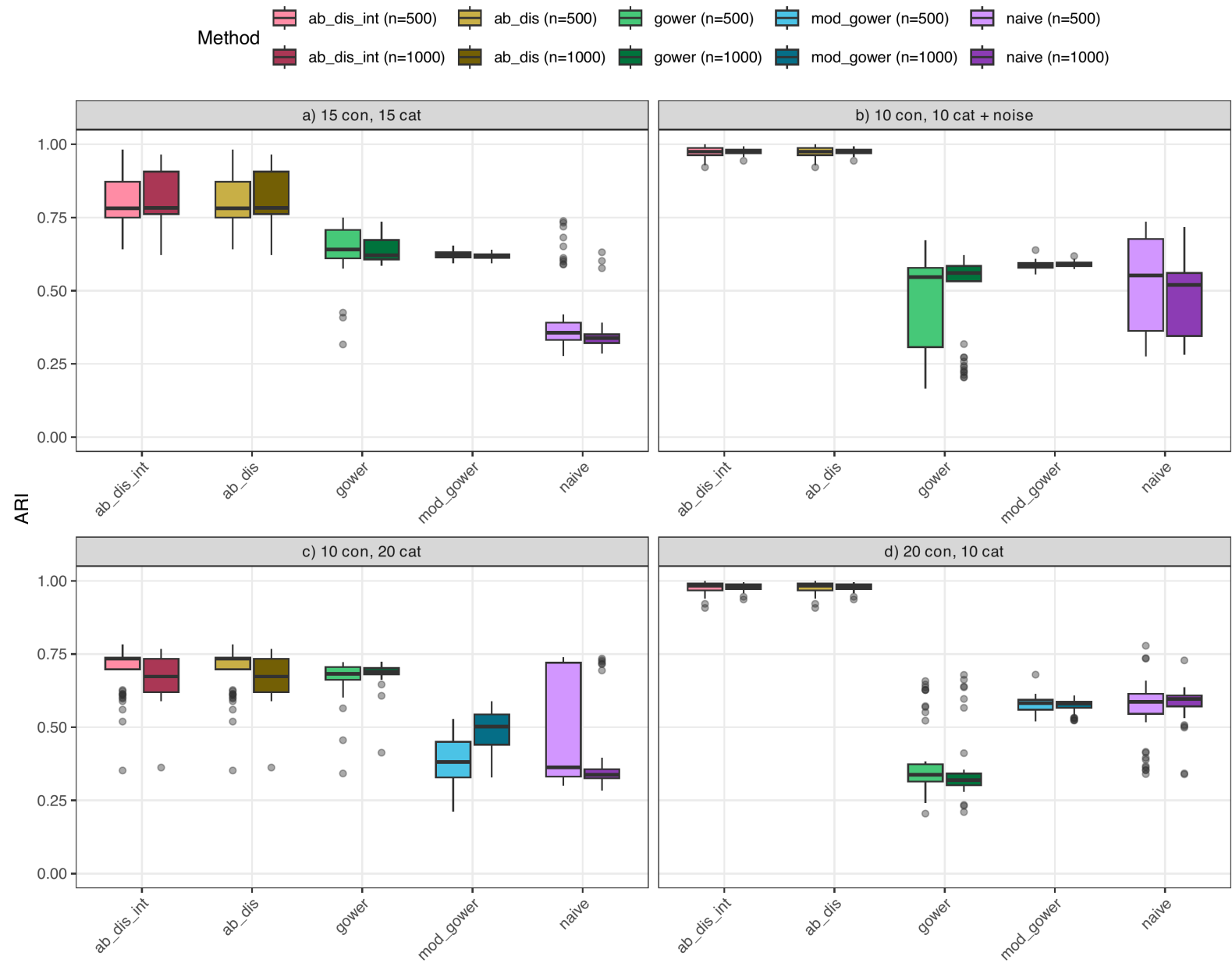
Simulation 1: associated mixed variables

Data-generating idea

1. Generate numerical variables from cluster-specific multivariate normal distributions.
 - different cluster means;
 - different correlation structures.
2. Generate binary categorical variables conditionally on the numerical variables.
 - the first categorical variable depends on the numerical block;
 - later categorical variables depend on the numerical block and on previous categorical variables.

the design induces numerical correlations, categorical associations, and numerical–categorical dependence

Simulation 1: results



Simulation 2: interaction-driven clusters

Design

- $I = 500, 1000$
- six continuous variables and three categorical variables
- V_4, V_5, V_6 generated independently from $N(0, 1)$
- V_1, V_2, V_3 generated conditionally on C_1 and C_2

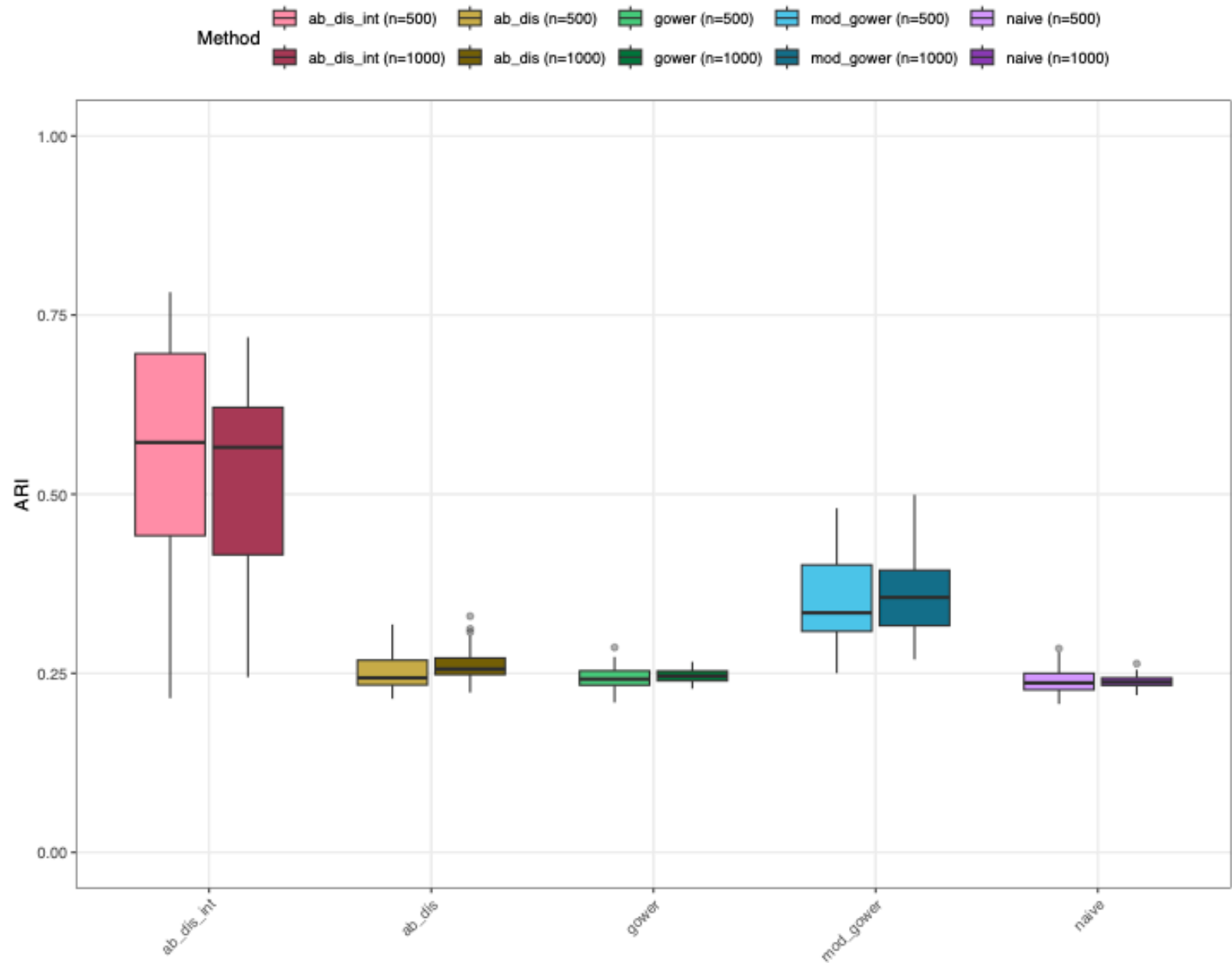
Main feature

The clusters are not defined by continuous variables alone or categorical variables alone.

They are defined by their **cross-type interaction**.

this is the setting where Δ^{int} should matter

Simulation 2: results



5. Response-aware extensions

Response-aware distances for KNN

KNN is usually described as a lazy learner:

- store the training data;
- compute distances from a new observation to the training observations;
- predict from the responses of the nearest neighbours.

Reframing KNN

The distance is not just a preprocessing choice.

It determines the neighbourhoods used for classification or regression.

in supervised learning, the response can help define these neighbourhoods

Response-aware mixed distance

For mixed-type predictors, use a supervised distance with two components:

$$D_{i\ell} = D^{(n)}(\mathbf{x}_i^{(n)}, \mathbf{x}_\ell^{(n)}) + D^{(c)}(\mathbf{x}_i^{(c)}, \mathbf{x}_\ell^{(c)}).$$

Numerical part

Use discriminant information from y to weight numerical differences.

Categorical part

Use the association between categories and y to define category dissimilarities.

Numerical part: discriminant weighting¹

For continuous predictors, use the response to weight directions or variables.

Single-variable discriminant weighting

For numerical variable j , define the Fisher score

$$\omega_j = \frac{B_j}{W_j},$$

where B_j and W_j are the between- and within-group variances.

Then a supervised Manhattan-type distance is

$$D^{(n)}(\mathbf{x}_i^{(n)}, \mathbf{x}_\ell^{(n)}) = \sum_{j=1}^{Q_n} \sqrt{\omega_j} |x_{ij}^{(n)} - x_{\ell j}^{(n)}|.$$

Categorical part: supervised TVD

For categorical predictors, compare categories through their response profiles.

Let $\mathbf{Z}_y$ be the indicator matrix of the response.

The supervised profile matrix is

$$\mathbf{R}_s = \mathbf{P}_d^{-1} \mathbf{Z}^\top \mathbf{Z}_y.$$

The supervised category dissimilarity is

$$\delta_s^j(a, b) = \frac{1}{2} \sum_{\ell=1}^{q_y} |\mathbf{r}_{a\ell}^{jy} - \mathbf{r}_{b\ell}^{jy}|.$$

categories are close if they show similar response distributions

Response-aware KNN: Carseats example

Data

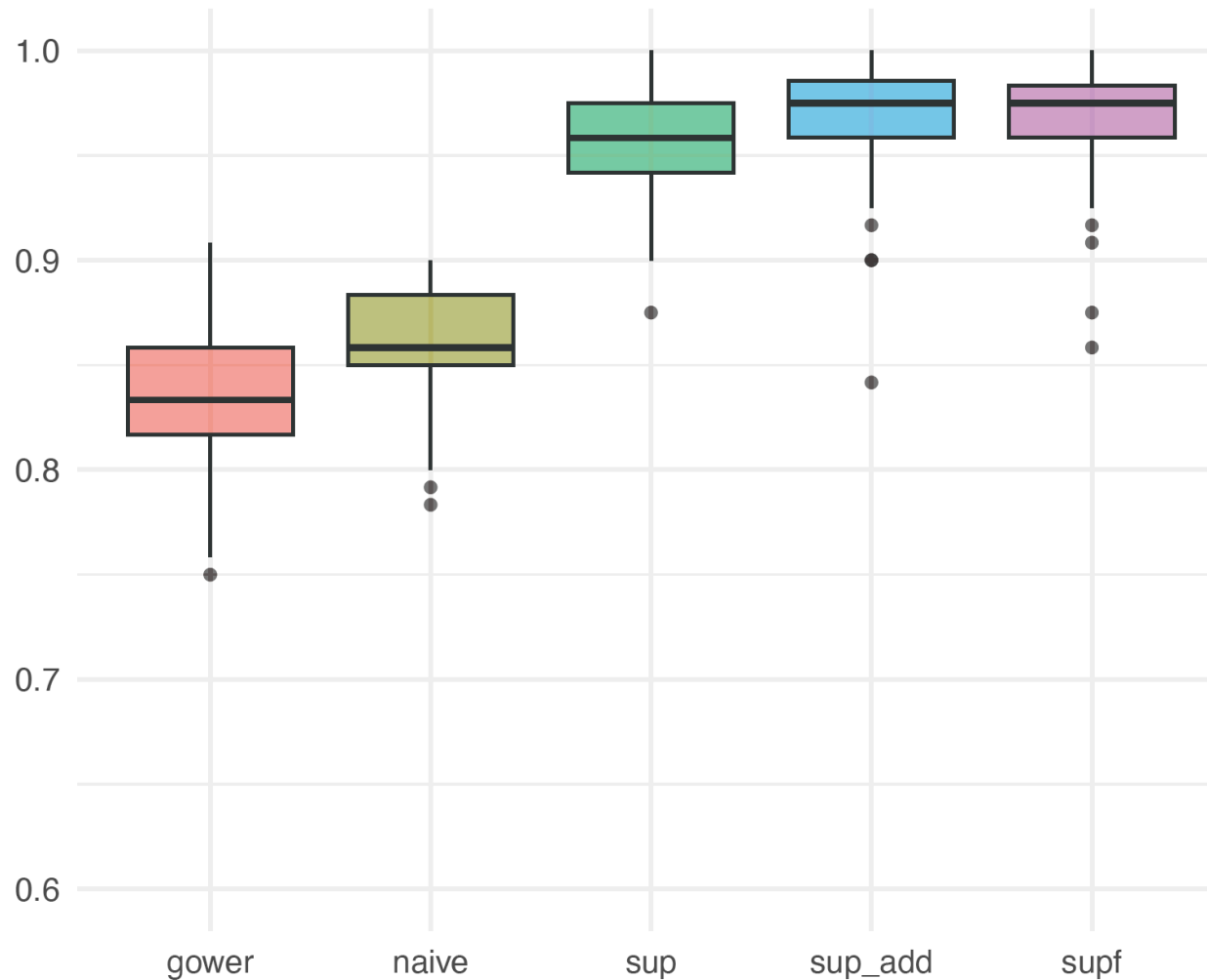
The `Carseats` data are used to predict whether sales are high.

- 7 numerical predictors
- 3 categorical predictors
- categorical predictors with 2 or 3 categories
- binary response: high vs. low sales

Compared distances

- `gower` : robust Manhattan + matching
- `naive` : Euclidean on scaled numerical variables and dummies
- `sup` : supervised numerical weighting + supervised TVD
- `sup_add` : additive supervised version
- `supf` : full supervised version

Response-aware KNN: Carseats results



- response-aware distances improve nearest-neighbour classification accuracy
- `sup`, `sup_add`, and `supf` are clearly above `gower` and `naive`
- the response helps define neighbourhoods that better reflect the class structure

Building distance-based pipelines



Building distance-based pipelines

manydist

A package to construct, diagnose, and use distances for continuous, categorical, and mixed-type data.



Distance construction

- `mdist()`
 - presets and custom specifications
 - response-aware distances
- `step_mdist()`
 - integrates distance construction into `tidymodels` workflows

Diagnostics

- `lovo_mdists()`
- `compare_lovo_mdists()`
- `benchmark_mdists()`

Learning: model specs

Unsupervised learning

- `pam_dist()`
- `spectral_dist()`

Supervised learning

- `nearest_neighbor_dist()`

manydist: socio-economic country profiles



Data

A 2022 World Bank / WDI snapshot of country-level socio-economic indicators.

- observations: countries
- numerical variables: GDP per capita, life expectancy, unemployment, urban population, population growth
- categorical variables: world region and income group

Use `manydist` to build a mixed-type distance between countries and diagnose which variables shape the resulting dissimilarity structure.

Preparing the WDI data

Country	Region	Income group	World Bank lending category	GDP per capita (k USD)	Life expectancy (years)	Unemployment (%)	Urban population (% total)	Population growth (%)
Tajikistan	Europe & Central Asia	Lower middle income	IDA	1.1	71.6	7.1	26.2	2.14
West Bank and Gaza	Middle East, North Africa, Afghanistan & Pakistan	Lower middle income	Not classified	3.8	76.7	24.4	86.6	2.43
Belarus	Europe & Central Asia	Upper middle income	IBRD	8.0	74.1	3.6	78.5	-0.80
United Arab Emirates	Middle East, North Africa, Afghanistan & Pakistan	High income	Not classified	50.8	80.5	2.9	85.5	5.09
El Salvador	Latin America & Caribbean	Upper middle income	IBRD	5.1	72.0	3.0	74.1	0.39
New Zealand	East Asia & Pacific	High income	Not classified	49.1	82.0	3.3	83.9	-0.06
Cyprus	Europe & Central Asia	High income	Not classified	33.2	80.4	6.8	66.7	1.06
Zambia	Sub-Saharan Africa	Lower middle income	IDA	1.4	65.3	6.0	44.6	2.76

Constructing a mixed-type distance using a preset



Directly with `mdist()`

```
1 wdi_x <- wdi_data |>
2   dplyr::select(-country)
3
4 d_preset <- mdist(
5   wdi_x,
6   preset = "u_dep"
7 )
```

Inside a recipe with `step_mdist()`

```
1 rec_preset <- recipe(~ ., data = wdi_x) |>
2   step_mdist(
3     all_predictors(),
4     preset = "u_dep"
5   )
6
7 d_preset_step <- rec_preset |>
8   prep(training = wdi_x) |>
9   bake(new_data = NULL)
```

```
1 all.equal(
2   unname(as.matrix(d_preset$distance)),
3   unname(as.matrix(d_preset_step))
4 )
```

[1] TRUE

`step_mdist()` embeds the same distance specification into a modelling workflow

Constructing a custom mixed-type distance



Directly with `mdist()`

```
1 wdi_x <- wdi_data |>
2   dplyr::select(-country)
3
4 d_custom <- mdist(
5   wdi_x,
6   method_cat = "eskin",
7   method_num = "std",
8   commensurable = FALSE
9 )
```

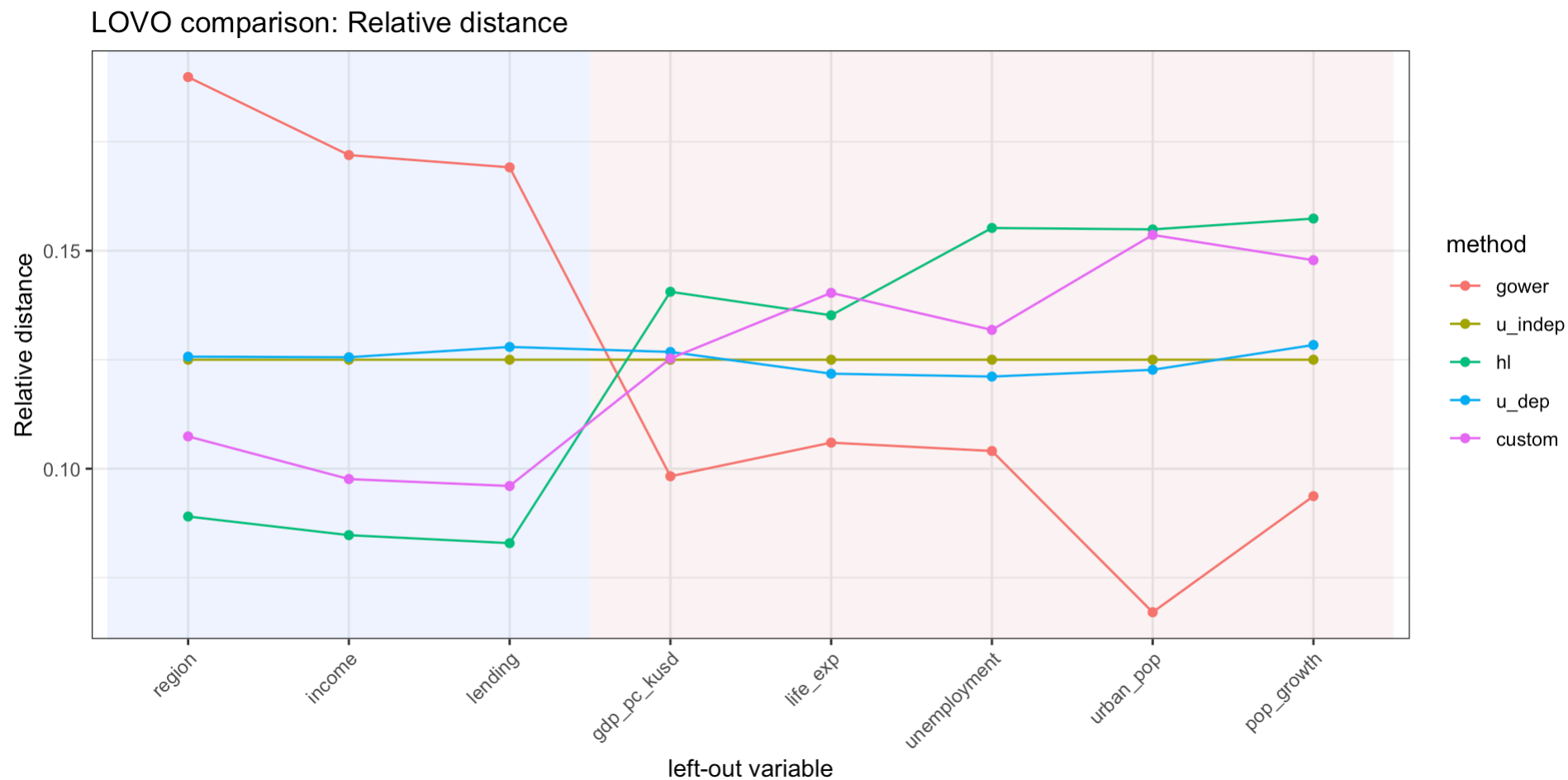
Inside a recipe with `step_mdists()`

```
1 rec_custom <- recipe(~ ., data = wdi_x) |>
2   step_mdists(
3     all_predictors(),
4     method_cat = "eskin",
5     method_num = "std",
6     commensurable = FALSE
7   )
8
9 d_custom_step <- rec_custom |>
10  prep(training = wdi_x) |>
11  bake(new_data = NULL)
```

```
1 all.equal(
2   unname(as.matrix(d_custom$distance)),
3   unname(as.matrix(d_custom_step))
4 )
```

[1] TRUE

Comparing distance constructions



```
lovo_mdists_compare()
```

The same leave-one-variable-out diagnostic can be computed for several distance definitions and compared in one display.

Distance-based classification pipeline

```
1 set.seed(123)
2 wdi_region <- wdi_data |>
3   dplyr::filter(region != "North America") |>
4   dplyr::mutate(
5     region = droplevels(region)
6   )
7 wdi_split <- initial_split(
8   wdi_region,
9   strata = region
10 )
11 wdi_train <- training(wdi_split)
12 wdi_test <- testing(wdi_split)
13 wdi_rec <- recipe(region ~ ., data = wdi_train) |>
14   update_role(country, new_role = "id") |>
15   step_mdists(
16     all_predictors(),
17     preset = "u_dep"
18   )
19 knn_spec <- nearest_neighbor_dist(
20   mode = "classification",
21   neighbors = tune()
22 )
23 wdi_wf <- workflow() |>
24   add_recipe(wdi_rec) |>
25   add_model(knn_spec)
```

Data

Prepare the classification task.



Distance-based classification pipeline

```
1 set.seed(123)
2 wdi_region <- wdi_data |>
3   dplyr::filter(region != "North America") |>
4   dplyr::mutate(
5     region = droplevels(region)
6   )
7 wdi_split <- initial_split(
8   wdi_region,
9   strata = region
10 )
11 wdi_train <- training(wdi_split)
12 wdi_test <- testing(wdi_split)
13 wdi_rec <- recipe(region ~ ., data = wdi_train) |>
14   update_role(country, new_role = "id") |>
15   step_mdist(
16     all_predictors(),
17     preset = "u_dep"
18   )
19 knn_spec <- nearest_neighbor_dist(
20   mode = "classification",
21   neighbors = tune()
22 )
23 wdi_wf <- workflow() |>
24   add_recipe(wdi_rec) |>
25   add_model(knn_spec)
```

Data

Prepare the classification task.



Split

Create training and test sets.

Distance-based classification pipeline

```
1 set.seed(123)
2 wdi_region <- wdi_data |>
3   dplyr::filter(region != "North America") |>
4   dplyr::mutate(
5     region = droplevels(region)
6   )
7 wdi_split <- initial_split(
8   wdi_region,
9   strata = region
10 )
11 wdi_train <- training(wdi_split)
12 wdi_test  <- testing(wdi_split)
13 wdi_rec <- recipe(region ~ ., data = wdi_train) |>
14   update_role(country, new_role = "id") |>
15   step_mdist(
16     all_predictors(),
17     preset = "u_dep"
18   )
19 knn_spec <- nearest_neighbor_dist(
20   mode = "classification",
21   neighbors = tune()
22 )
23 wdi_wf <- workflow() |>
24   add_recipe(wdi_rec) |>
25   add_model(knn_spec)
```



Data

Prepare the classification task.

Split

Create training and test sets.

Recipe

Use `step_mdist()` to construct the distance representation.

Distance-based classification pipeline

```
1 set.seed(123)
2 wdi_region <- wdi_data |>
3   dplyr::filter(region != "North America") |>
4   dplyr::mutate(
5     region = droplevels(region)
6   )
7 wdi_split <- initial_split(
8   wdi_region,
9   strata = region
10 )
11 wdi_train <- training(wdi_split)
12 wdi_test <- testing(wdi_split)
13 wdi_rec <- recipe(region ~ ., data = wdi_train) |>
14   update_role(country, new_role = "id") |>
15   step_mdist(
16     all_predictors(),
17     preset = "u_dep"
18   )
19 knn_spec <- nearest_neighbor_dist(
20   mode = "classification",
21   neighbors = tune()
22 )
23 wdi_wf <- workflow() |>
24   add_recipe(wdi_rec) |>
25   add_model(knn_spec)
```



Data

Prepare the classification task.

Split

Create training and test sets.

Recipe

Use `step_mdist()` to construct the distance representation.

Model

Specify a distance-based KNN classifier.

Distance-based classification pipeline

```
1 set.seed(123)
2 wdi_region <- wdi_data |>
3   dplyr::filter(region != "North America") |>
4   dplyr::mutate(
5     region = droplevels(region)
6   )
7 wdi_split <- initial_split(
8   wdi_region,
9   strata = region
10 )
11 wdi_train <- training(wdi_split)
12 wdi_test <- testing(wdi_split)
13 wdi_rec <- recipe(region ~ ., data = wdi_train) |>
14   update_role(country, new_role = "id") |>
15   step_mdists(
16     all_predictors(),
17     preset = "u_dep"
18   )
19 knn_spec <- nearest_neighbor_dist(
20   mode = "classification",
21   neighbors = tune()
22 )
23 wdi_wf <- workflow() |>
24   add_recipe(wdi_rec) |>
25   add_model(knn_spec)
```



Data

Prepare the classification task.

Split

Create training and test sets.

Recipe

Use `step_mdists()` to construct the distance representation.

Model

Specify a distance-based KNN classifier.

Workflow

Combine preprocessing and model specification.

Tuning a distance-based KNN pipeline

```
1 set.seed(123)
2 wdi_folds <- vfold_cv(
3   wdi_train,
4   v = 5,
5   strata = region
6 )
7 knn_grid <- tibble(
8   neighbors = c(1, 3, 5, 7, 9, 11, 15)
9 )
10 knn_tuned <- tune_grid(
11   wdi_wf,
12   resamples = wdi_folds,
13   grid = knn_grid,
14   metrics = metric_set(accuracy)
15 )
16 best_k <- select_best(
17   knn_tuned,
18   metric = "accuracy"
19 )
20 final_wf <- finalize_workflow(
21   wdi_wf,
22   best_k
23 )
24 final_res <- last_fit(
25   final_wf,
26   split = wdi_split,
27   metrics = metric_set(accuracy)
28 )
```

Resample

Create cross-validation folds on the training set.



Tuning a distance-based KNN pipeline

```
1 set.seed(123)
2 wdi_folds <- vfold_cv(
3   wdi_train,
4   v = 5,
5   strata = region
6 )
7 knn_grid <- tibble(
8   neighbors = c(1, 3, 5, 7, 9, 11, 15)
9 )
10 knn_tuned <- tune_grid(
11   wdi_wf,
12   resamples = wdi_folds,
13   grid = knn_grid,
14   metrics = metric_set(accuracy)
15 )
16 best_k <- select_best(
17   knn_tuned,
18   metric = "accuracy"
19 )
20 final_wf <- finalize_workflow(
21   wdi_wf,
22   best_k
23 )
24 final_res <- last_fit(
25   final_wf,
26   split = wdi_split,
27   metrics = metric_set(accuracy)
28 )
```

Resample

Create cross-validation folds on the training set.



Grid

Define candidate values for the number of neighbours.

Tuning a distance-based KNN pipeline

```
1 set.seed(123)
2 wdi_folds <- vfold_cv(
3   wdi_train,
4   v = 5,
5   strata = region
6 )
7 knn_grid <- tibble(
8   neighbors = c(1, 3, 5, 7, 9, 11, 15)
9 )
10 knn_tuned <- tune_grid(
11   wdi_wf,
12   resamples = wdi_folds,
13   grid = knn_grid,
14   metrics = metric_set(accuracy)
15 )
16 best_k <- select_best(
17   knn_tuned,
18   metric = "accuracy"
19 )
20 final_wf <- finalize_workflow(
21   wdi_wf,
22   best_k
23 )
24 final_res <- last_fit(
25   final_wf,
26   split = wdi_split,
27   metrics = metric_set(accuracy)
28 )
```

Resample

Create cross-validation folds on the training set.



Grid

Define candidate values for the number of neighbours.

Tune

Evaluate each candidate value by cross-validation.

Tuning a distance-based KNN pipeline

```
1 set.seed(123)
2 wdi_folds <- vfold_cv(
3   wdi_train,
4   v = 5,
5   strata = region
6 )
7 knn_grid <- tibble(
8   neighbors = c(1, 3, 5, 7, 9, 11, 15)
9 )
10 knn_tuned <- tune_grid(
11   wdi_wf,
12   resamples = wdi_folds,
13   grid = knn_grid,
14   metrics = metric_set(accuracy)
15 )
16 best_k <- select_best(
17   knn_tuned,
18   metric = "accuracy"
19 )
20 final_wf <- finalize_workflow(
21   wdi_wf,
22   best_k
23 )
24 final_res <- last_fit(
25   final_wf,
26   split = wdi_split,
27   metrics = metric_set(accuracy)
28 )
```

Resample

Create cross-validation folds on the training set.



Grid

Define candidate values for the number of neighbours.

Tune

Evaluate each candidate value by cross-validation.

Select

Choose the best-performing number of neighbours.

Tuning a distance-based KNN pipeline

```
1 set.seed(123)
2 wdi_folds <- vfold_cv(
3   wdi_train,
4   v = 5,
5   strata = region
6 )
7 knn_grid <- tibble(
8   neighbors = c(1, 3, 5, 7, 9, 11, 15)
9 )
10 knn_tuned <- tune_grid(
11   wdi_wf,
12   resamples = wdi_folds,
13   grid = knn_grid,
14   metrics = metric_set(accuracy)
15 )
16 best_k <- select_best(
17   knn_tuned,
18   metric = "accuracy"
19 )
20 final_wf <- finalize_workflow(
21   wdi_wf,
22   best_k
23 )
24 final_res <- last_fit(
25   final_wf,
26   split = wdi_split,
27   metrics = metric_set(accuracy)
28 )
```

Resample

Create cross-validation folds on the training set.



Grid

Define candidate values for the number of neighbours.

Tune

Evaluate each candidate value by cross-validation.

Select

Choose the best-performing number of neighbours.

Finalize

Insert the selected value into the workflow.

Tuning a distance-based KNN pipeline

```
1 set.seed(123)
2 wdi_folds <- vfold_cv(
3   wdi_train,
4   v = 5,
5   strata = region
6 )
7 knn_grid <- tibble(
8   neighbors = c(1, 3, 5, 7, 9, 11, 15)
9 )
10 knn_tuned <- tune_grid(
11   wdi_wf,
12   resamples = wdi_folds,
13   grid = knn_grid,
14   metrics = metric_set(accuracy)
15 )
16 best_k <- select_best(
17   knn_tuned,
18   metric = "accuracy"
19 )
20 final_wf <- finalize_workflow(
21   wdi_wf,
22   best_k
23 )
24 final_res <- last_fit(
25   final_wf,
26   split = wdi_split,
27   metrics = metric_set(accuracy)
28 )
```

Resample

Create cross-validation folds on the training set.



Grid

Define candidate values for the number of neighbours.

Tune

Evaluate each candidate value by cross-validation.

Select

Choose the best-performing number of neighbours.

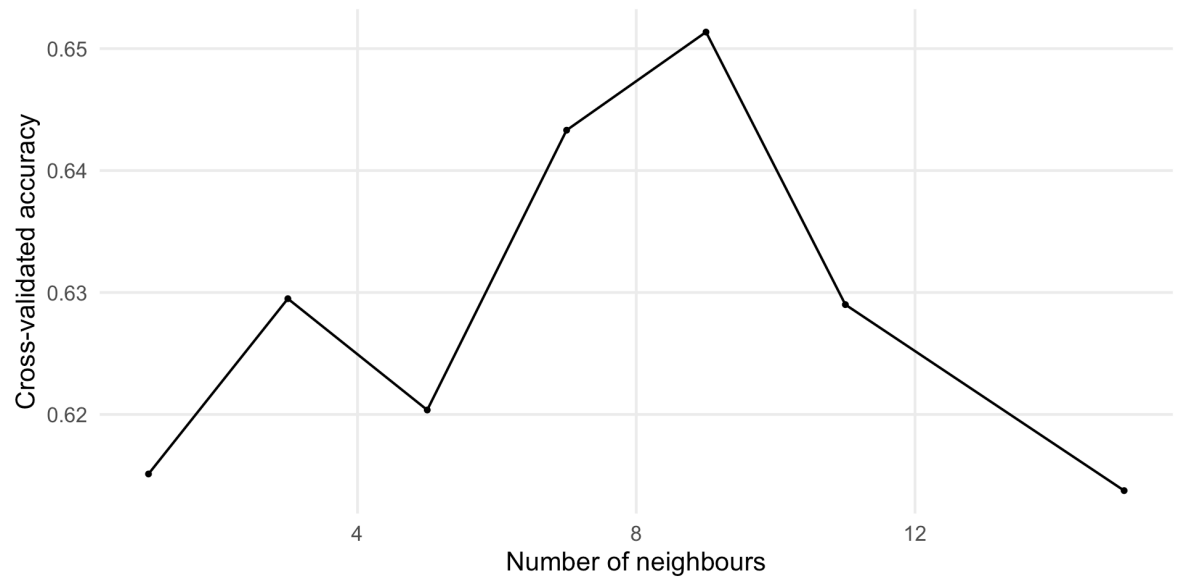
Finalize

Insert the selected value into the workflow.

Test

Fit the finalized workflow on the training set and evaluate it on the test set.

Tuning results



Test-set performance

Metric	Accuracy
accuracy	0.553

Workflow

`last_fit()` fits the finalized workflow on the full training set and evaluates it once on the held-out test set.

Wrap-up

Increasing awareness works in two directions.

Distances must be data-aware

Construct distances that reflect the structure of the data they are applied to, including variable types, associations, and interactions.

Practitioners must be distance-aware

Understand what a chosen distance emphasizes or suppresses, and how that choice shapes downstream results.

A package makes this practical

manydist makes distance construction easy to use in learning pipelines. During resampling, `step_mdist()` fits the transformation on each analysis fold and applies it to the assessment fold, helping avoid data leakage.

Main references

- Iodice D’Enza, A., Tortora, C., & Palumbo, F. (2026). Association-based spectral clustering for mixed data with cross-type interactions. *Manuscript Submitted to Statistics and Computing*.
- van de Velden, M., Iodice D’Enza, A., Markos, A., & Cavicchia, C. (2026). A general framework for unbiased mixed-variables distances. *Under Review: Journal of Computational and Graphical Statistics*.
- Ghashti, J. S., & Thompson, J. R. J. (2025). Mixed-type distance shrinkage and selection for clustering via kernel metric learning. *Journal of Classification*, 42(2), 311–334.
- Liu, P., Yuan, H., Ning, Y., Chakraborty, B., Liu, N., & Peres, M. A. (2024). A modified and weighted gower distance-based clustering analysis for mixed type data: A simulation and empirical analyses. *BMC Medical Research Methodology*, 24(305).
- van de Velden, M., Iodice D’Enza, A., Markos, A., & Cavicchia, C. (2024). A general framework for implementing distances for categorical variables. *Pattern Recognition*, 153, 110547.
- Mousavi, E., & Sehati, M. (2023). A generalized multi-aspect distance metric for mixed-type data clustering. *Pattern Recognition*, 109353.
- Hennig, Christian, Viroli, C., & Anderlucci, L. (2019). Quantile-based clustering. *Electronic Journal of Statistics*, 13(2), 4849–4883. <https://doi.org/10.1214/19-EJS1640>
- Hennig, C., & Liao, T. F. (2013). How to Find an Appropriate Clustering for Mixed-Type Variables with Application to Socio-Economic Stratification. *Journal of the Royal Statistical Society Series C: Applied Statistics*, 62(3), 309–369.
- Kaufman, L., & Rousseeuw, P. J. (2009). *Finding groups in data: An introduction to cluster analysis*. John Wiley & Sons.
- Ben-Israel, A., & Iyigun, C. (2008). Probabilistic d-clustering. *Journal of Classification*, 25(1), 5–26.
- Borg, I., & Groenen, P. J. (2005). *Modern multidimensional scaling: Theory and applications*. Springer.
- Le, S. Q., & Ho, T. B. (2005). An association-based dissimilarity measure for categorical data. *Pattern Recognition Letters*, 26(16), 2549–2557.
- Ng, A. Y., Jordan, M. I., & Weiss, Y. (2002). On spectral clustering: Analysis and an algorithm. *Advances in Neural Information Processing Systems*, 14, 849–856. MIT Press.
- Ng, A., Jordan, M., & Weiss, Y. (2001). On spectral clustering: Analysis and an algorithm. *Advances in Neural Information Processing Systems*, 14.
- Hastie, T., & Tibshirani, R. (1995). Discriminant adaptive nearest neighbor classification and regression. *Advances in Neural Information Processing Systems*, 8. MIT Press.

Contact and slides

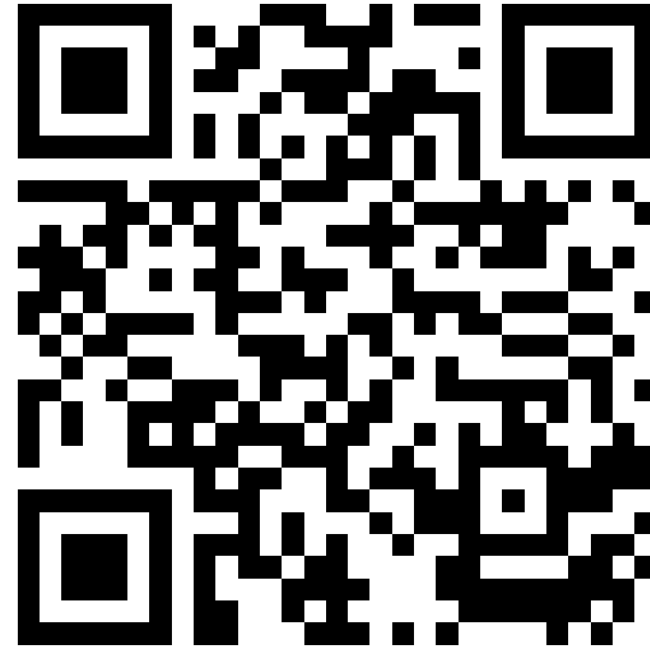
Contact

Alfonso Iodice D'Enza
iodicede@unina.it

GitHub Pages

<https://alfonsoiodicede.github.io>

scan to open the package website



[Package website](#)

Backup: distance–density derivation

Just one-way interaction?

Factorization motivates the direction

Let $\mathbf{x}_i = (\mathbf{x}_{i_n}, \mathbf{x}_{i_c})$.

The joint distribution can always be written as

$$f(\mathbf{x}_{i_n}, \mathbf{x}_{i_c}) = f(\mathbf{x}_{i_n})f(\mathbf{x}_{i_c} \mid \mathbf{x}_{i_n}).$$

This motivates using the continuous variables to quantify categorical distinctions:

$$\mathbf{X}_n \longrightarrow \delta_{int}^j(a, b).$$

How can this result be connected to pairwise dissimilarities?

Just one-way interaction?

Dissimilarity and reference density

Starting from a non-negative dissimilarity from a prototype,

$$d_{\mathbf{S}_k}(\mathbf{x}_i, \mathbf{c}_k) \geq 0, \quad d_{\mathbf{S}_k}(\mathbf{c}_k, \mathbf{c}_k) = 0,$$

a distance-based probability model can be constructed, provided that the exponential kernel is normalisable ¹:

$$f(\mathbf{x}_i; \mathbf{c}_k, \mathbf{S}_k) = g(\mathbf{c}_k, \mathbf{S}_k, \eta) \exp \{-\eta d_{\mathbf{S}_k}(\mathbf{x}_i, \mathbf{c}_k)\}, \quad \eta > 0,$$

where $g(\mathbf{c}_k, \mathbf{S}_k, \eta)$ makes f a proper density.

Just one-way interaction?

Dissimilarity and reference density

Since the density is maximal at the reference point,

$$M_k = \max_{\mathbf{x}} f(\mathbf{x}; \mathbf{c}_k, \mathbf{S}_k) = f(\mathbf{c}_k; \mathbf{c}_k, \mathbf{S}_k) = g(\mathbf{c}_k, \mathbf{S}_k, \eta).$$

the **maximum-normalised kernel** is

$$\bar{K}_k(\mathbf{x}_i, \mathbf{c}_k) = \frac{f(\mathbf{x}_i; \mathbf{c}_k, \mathbf{S}_k)}{M_k} = \exp \{ -\eta d_{\mathbf{S}_k}(\mathbf{x}_i, \mathbf{c}_k) \}.$$

Therefore,

$$d_{\mathbf{S}_k}(\mathbf{x}_i, \mathbf{c}_k) = -\frac{1}{\eta} \log \bar{K}_k(\mathbf{x}_i, \mathbf{c}_k).$$

Just one-way interaction?

The pairwise continuous component

For pairwise comparison, let every observation act in turn as the reference prototype:

$$\mathbf{c}_k \longrightarrow \mathbf{x}_{i'_n}, \quad \mathbf{S}_k \longrightarrow \mathbf{S}.$$

Use the ordinary Mahalanobis distance and choose $\eta = 1$:

$$f_n(\mathbf{x}_{i_n}; \mathbf{x}_{i'_n}, \mathbf{S}) = g_n(\mathbf{S}) \exp \{ -d_{\text{mah}}(\mathbf{x}_{i_n}, \mathbf{x}_{i'_n}) \}.$$

Using the reference maximum $M_n = g_n(\mathbf{S})$ from the preceding construction

the **maximum-normalised continuous kernel** is

$$\overline{K}_n(i, i') = \frac{f_n(\mathbf{x}_{i_n}; \mathbf{x}_{i'_n}, \mathbf{S})}{M_n} = \exp \{ -d_{\text{mah}}(\mathbf{x}_{i_n}, \mathbf{x}_{i'_n}) \}.$$

$-\log \overline{K}_n(i, i')$ recovers the ordinary Mahalanobis distance

Just one-way interaction?

The pairwise interaction-aware categorical component

For categorical variable j , let $x_{ij_c} = a$ and $x_{i'j_c} = b$.

The interaction-aware category dissimilarity is

$$\tilde{\delta}^j(a, b) = (1 - \alpha)\delta_{tvd}^j(a, b) + \alpha\delta_{int}^j(a, b).$$

Assume that $\tilde{\delta}^j(a, b) \geq 0$ and $\tilde{\delta}^j(b, b) = 0$.

Choosing $\eta_j = 1$, define the categorical reference PMF

$$p_j(a; b) = \text{softmax}_{h \in \mathcal{A}_j} \left\{ -\tilde{\delta}^j(h, b) \right\}_a = \frac{\exp \left\{ -\tilde{\delta}^j(a, b) \right\}}{\sum_{h \in \mathcal{A}_j} \exp \left\{ -\tilde{\delta}^j(h, b) \right\}}.$$

smaller category dissimilarities receive larger reference probabilities

Just one-way interaction?

The pairwise interaction-aware categorical component

Because $\tilde{\delta}^j(b, b) = 0$, the maximum reference probability is

$$M_j(b) = p_j(b; b) = \frac{1}{\sum_{h \in \mathcal{A}_j} \exp \left\{ -\tilde{\delta}^j(h, b) \right\}}.$$

The **maximum-normalised categorical kernel**

$$\overline{K}_j(a, b) = \frac{p_j(a; b)}{M_j(b)} = \exp \left\{ -\tilde{\delta}^j(a, b) \right\},$$

$-\log \overline{K}_j(a, b) = \tilde{\delta}^j(a, b)$ recovers the interaction-aware dissimilarity

Just one-way interaction?

Combining the kernels

For the whole categorical vector, define the separable maximum-normalised kernel

$$\begin{aligned}\overline{K}_c^{(int)}(i, i') &= \prod_{j=1}^{Q_c} \overline{K}_j(x_{ij_c}, x_{i'j_c}) \\ &= \exp \left\{ - \sum_{j=1}^{Q_c} \tilde{\delta}^j(x_{ij_c}, x_{i'j_c}) \right\} \\ &= \exp \left\{ -d_c^{(int)}(\mathbf{x}_{i_c}, \mathbf{x}_{i'_c}) \right\}.\end{aligned}$$

Just one-way interaction?

Combining the kernels

Combine the continuous and categorical kernels:

$$\overline{K}_{\text{mix}}^{(int)}(i, i') = \overline{K}_n(i, i') \overline{K}_c^{(int)}(i, i') = \exp \left\{ -d_{\text{mah}}(\mathbf{x}_{i_n}, \mathbf{x}_{i'_n}) - d_c^{(int)}(\mathbf{x}_{i_c}, \mathbf{x}_{i'_c}) \right\}$$

$$-\log \overline{K}_{\text{mix}}^{(int)}(i, i') = d_{\text{mah}}(\mathbf{x}_{i_n}, \mathbf{x}_{i'_n}) + d_c^{(int)}(\mathbf{x}_{i_c}, \mathbf{x}_{i'_c})$$

recovers the overall dissimilarity

in algebraic form

$$\mathbf{D}_{\text{mix}}^{(int)} = \mathbf{D}_{\text{mah}} + \mathbf{Z} \tilde{\Delta} \mathbf{Z}^\top.$$